

INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DA PARAÍBA
CAMPUS CAMPINA GRANDE

JOSÉ FELLIPE BERNARDINO LIMA

PROCESSAMENTO DIGITAL DE SINAIS
MOACY PEREIRA DA SILVA

RELATÓRIO ESTUDO DIRIGIDO - PARTE 1

INTRODUÇÃO

Este estudo dirigido tem como objetivo abordar tanto a conceitualização do significado de sinais quanto na prática, simulando funções e discutindo os resultados obtidos a partir destas. Neste relatório, serão abordados tópicos que introduzem a disciplina de Processamento Digital de Sinais: Sinais Contínuos e Discretos, Amostragem, Quantização e Resolução, Sequências Fundamentais, Operações com Sinais, Energia e Potência, Sistemas Discretos, Sistemas LTI, Convolução Discreta, Convolução como Filtragem. Também será desenvolvido um mini projeto de um Sistema de Aquisição e Processamento de Sinais para integrar os principais conceitos explorados nessa Parte 1.

ESTUDO DOS TÓPICOS

1. Sinais Contínuos e Discretos

No contexto da engenharia, um sinal é qualquer grandeza física ou variável que carrega alguma informação sobre o estado ou comportamento de um sistema e que varia em função de uma ou mais variáveis independentes (como tempo, espaço, temperatura, etc.). Na maioria das aplicações de Processamento Digital de Sinais (PDS), a variável independente principal é o tempo.

Os sinais podem ser classificados, quanto à natureza da variável independente, em dois grandes grupos:

- Sinais Contínuos no Tempo: São sinais definidos para qualquer instante de tempo dentro de um intervalo. O tempo flui de maneira contínua, o que significa que entre dois instantes quaisquer existe uma infinidade de valores. Eles representam fenômenos da natureza em sua forma bruta e ininterrupta.
- Exemplos reais: A temperatura ambiente ao longo de um dia contínuo; a tensão elétrica gerada por um microfone dinâmico captando um áudio (onda de pressão sonora acústica).
- Sinais Discretos no Tempo: São sinais definidos apenas em instantes de tempo específicos e separados por intervalos (geralmente regulares). O eixo do tempo é "quebrado" em pedaços, formando uma sequência de valores. Muitas vezes, sinais discretos surgem da amostragem de um sinal contínuo.
- Exemplos reais: O áudio digital armazenado em um formato MP3 ou CD (onde a onda sonora foi capturada a uma taxa de 44.100 vezes por segundo); o registro diário do fechamento da bolsa de valores ou a medição de luminosidade capturada por um sensor digital de hora em hora.

Um fenômeno físico pode ser representado matematicamente associando a variação dessa grandeza física a uma função matemática. Por exemplo, a oscilação de um pêndulo (posição) no tempo pode ser mapeada para uma função senoidal, permitindo análises preditivas, transformações e simulações.

1.2. Formulação matemática

Um sinal contínuo é denotado como uma função de uma variável contínua "t" (tempo), pertencente ao conjunto dos números reais (R):

$x(t)$, onde t pertence aos Reais.

Um sinal discreto é representado como uma sequência de valores indexados por uma variável inteira "n", pertencente ao conjunto dos números inteiros (Z):

$x[n]$, onde n pertence aos Inteiros.

A relação direta entre o mundo contínuo e o discreto ocorre através do processo de amostragem. Se coletarmos amostras de um sinal contínuo $x(t)$ a cada intervalo de tempo constante T_s (período de amostragem), obtemos o sinal discreto:

$$x[n] = x(n * T_s)$$

Onde a frequência de amostragem (ou taxa de amostragem) é dada por $F_s = 1 / T_s$.

1.3. Exemplo

Vamos modelar uma tensão elétrica que varia senoidalmente, simulando um sinal captado por um sensor de vibração mecânica acoplado a um motor. Suponha que a vibração possua uma amplitude de 5 V e oscile a uma frequência de 2 Hz.

- Representação contínua:
- $x(t) = 5 * \text{sen}(2 * \pi * 2 * t)$
- Representação discreta:
- Vamos digitalizar esse sinal utilizando um conversor analógico-digital (ADC) operando com uma frequência de amostragem de $F_s = 20$ Hz. O período de amostragem será $T_s = 1 / 20 = 0.05$ s. O sinal discreto resultante será:
- $x[n] = 5 * \text{sen}(2 * \pi * 2 * n * 0.05)$
- $x[n] = 5 * \text{sen}(0.2 * \pi * n)$

1.4. Simulação

O script em Python a seguir utiliza as bibliotecas NumPy para a geração dos dados e Matplotlib para a plotagem dos gráficos, evidenciando as diferenças visuais entre os dois sinais.

```
import numpy as np
import matplotlib.pyplot as plt

# Parâmetros do sinal
f = 2.0          # Frequência do sinal em Hz (2 oscilações por segundo)
A = 5.0          # Amplitude
Fs = 20.0        # Frequência de amostragem em Hz
Ts = 1 / Fs      # Período de amostragem
```

```

# Sinal contínuo (Simulado usando uma alta resolução temporal)
t_cont = np.arange(0, 1.0, 0.001) # de 0 a 1 segundo com passos de 1
ms
x_cont = A * np.sin(2 * np.pi * f * t_cont)

# Sinal discreto (Amostrado)
n = np.arange(0, 20) # 20 amostras (cobre 1 segundo, pois
Fs = 20)
t_disc = n * Ts # Mapeando o índice n para o tempo
real em segundos
x_disc = A * np.sin(2 * np.pi * f * t_disc)

# --- Plotagem dos Resultados ---
plt.figure(figsize=(10, 6))

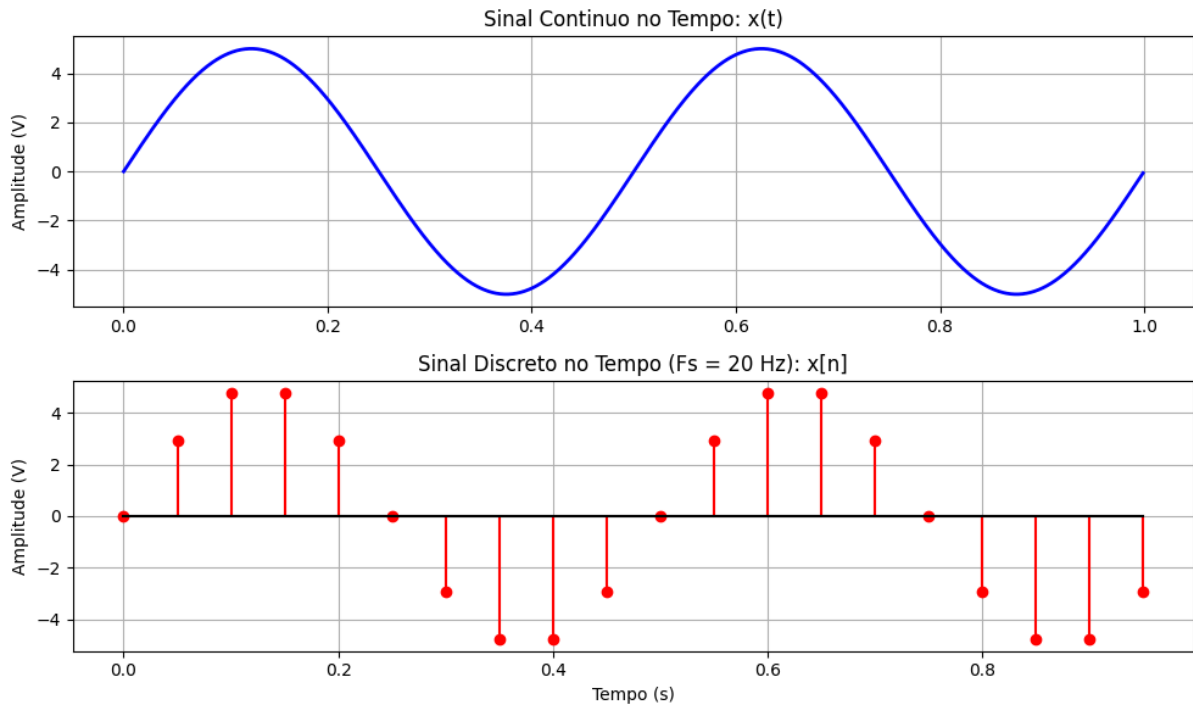
# Gráfico do Sinal Contínuo
plt.subplot(2, 1, 1)
plt.plot(t_cont, x_cont, color='blue', linewidth=2)
plt.title('Sinal Contínuo no Tempo: x(t)')
plt.ylabel('Amplitude (V)')
plt.grid(True)

# Gráfico do Sinal Discreto
plt.subplot(2, 1, 2)
# Utiliza-se 'stem' para representar sequências discretas graficamente
plt.stem(t_disc, x_disc, linefmt='red', markerfmt='ro', basefmt='k')
plt.title('Sinal Discreto no Tempo (Fs = 20 Hz): x[n]')
plt.xlabel('Tempo (s)')
plt.ylabel('Amplitude (V)')
plt.grid(True)

plt.tight_layout()
plt.show()

# Imprimindo os primeiros 5 valores da sequência discreta para análise
numérica
print("Índice (n) | Tempo (s) | Valor x[n]")
print("-" * 35)
for i in range(5):
    print(f"        {n[i]}          |      {t_disc[i]:.2f}      |      {x_disc[i]:.4f}")

```



1.5. Resultados

A execução do código gera uma janela gráfica com dois gráficos e a seguinte saída numérica no terminal para as 5 primeiras amostras do sinal discreto:

Índice (n)	Tempo (t) em s	Valor $x[n]$ em V
0	0.00	0.0000
1	0.05	2.9389
2	0.10	4.7553
3	0.15	4.7553
4	0.20	2.9389

Visualmente, a interface gerará:

1. Sinal Contínuo: Uma linha azul suave e ininterrupta conectando todos os pontos da função senoidal, completando dois ciclos inteiros ao longo de 1 segundo (já que a frequência $f = 2$ Hz).
2. Sinal Discreto: Um gráfico do tipo "haste" (stem plot), contendo exatamente 20 "palitos" verticais vermelhos encabeçados por círculos. O eixo horizontal exibe intervalos vazios entre as marcações de tempo (0.05s, 0.10s, etc.), refletindo a ausência de informação nos instantes entre as amostras.

1.6. Discussão

Os resultados expõem claramente a diferença na natureza das duas representações. No gráfico do sinal $x(t)$, assume-se que se quisermos saber a tensão em um instante arbitrário, como $t = 0.0734$ s, a função possui um valor exato definido para ele. Matematicamente, a variável de tempo opera no domínio dos números reais, permitindo precisão temporal infinita (limitada, em um computador, apenas pela resolução, por isso na simulação usamos um passo muito pequeno de 0.001 s para "fingir" o contínuo).

Por outro lado, o sinal discreto $x[n]$ não sabe o que acontece entre as amostras. O tempo $t = 0.0734$ s não existe na sequência $x[n]$; a informação saltou do instante $n=1$ ($t=0.05$ s) diretamente para $n=2$ ($t=0.10$ s). Essa é a essência de como os computadores lidam com sinais físicos: eles não conseguem processar variáveis perfeitamente contínuas, sendo necessário discretizá-las no tempo por meio da amostragem (e em amplitude por meio da quantização).

Apesar de perder os dados intermediários, o teorema da amostragem de Nyquist-Shannon garante que, desde que a taxa de amostragem (20 Hz) seja pelo menos o dobro da maior frequência presente no sinal de origem (2 Hz), a informação original (frequência, fase e amplitude da vibração) é integralmente preservada e pode ser reconstruída a partir das amostras discretas.

2.1. Amostragem

A amostragem é o processo fundamental que nos permite fazer a ponte entre o mundo físico (analogico e contínuo) e o mundo digital (discreto).

O significado físico da amostragem é equivalente a tirar várias "fotografias" de um fenômeno que está acontecendo ao longo do tempo. Em vez de observarmos e registrarmos a grandeza continuamente sem interrupções, nós "piscamos" os olhos e anotamos o valor do sinal apenas em instantes específicos e pré-determinados. O que acontece no intervalo de tempo entre uma anotação e outra é simplesmente ignorado pelo sistema digital.

2.2. Formulação matemática

A relação entre o sinal contínuo $x(t)$ e o sinal discreto amostrado $x[n]$ é dada pela equação:

$$x[n] = x(n * Ts)$$

Onde:

- n : É o número inteiro que representa o índice da amostra (0, 1, 2, 3...).
- Ts : É o período de amostragem, ou seja, o intervalo de tempo fixo (em segundos) que esperamos entre a coleta de uma amostra e a próxima.
- $t_n = n * Ts$: Representa o instante de tempo real (em segundos) em que a n -ésima amostra foi coletada.

A relação entre o período de amostragem (Ts) e a frequência de amostragem (fs) é inversamente proporcional, dada por:

$$fs = 1 / Ts$$

Enquanto Ts nos diz "de quanto em quanto tempo" coletamos uma amostra, a frequência fs nos diz "quantas amostras coletamos dentro de 1 segundo". A unidade de fs é o Hertz (Hz) ou amostras por segundo.

2.3. Exemplo

Para demonstrar esse conceito, vamos gerar um sinal contínuo senoidal com uma frequência de oscilação de $f = 5$ Hz (5 ciclos completos por segundo) e amplitude 1.

$$x(t) = \sin(2 * \pi * 5 * t)$$

Para investigar o efeito da amostragem, vamos "fotografar" esse sinal usando três frequências de amostragem diferentes:

- $fs1 = 100$ Hz ($Ts = 0.010$ s): Uma amostragem muito rápida.
- $fs2 = 25$ Hz ($Ts = 0.040$ s): Uma amostragem intermediária.
- $fs3 = 8$ Hz ($Ts = 0.125$ s): Uma amostragem lenta (perto da frequência do próprio sinal).

2.4. Simulação

O script em Python abaixo gera o sinal contínuo e plota, no mesmo gráfico, o sinal original e suas respectivas amostras discretas para as três frequências escolhidas.

```
import numpy as np
import matplotlib.pyplot as plt

# Parâmetros do sinal contínuo
f = 5.0 # Frequência do sinal (Hz)
t_max = 0.5 # Tempo total de simulação (segundos)
```

```

# 1. Gerando o sinal contínuo (alta resolução para simular o tempo contínuo)
t_cont = np.arange(0, t_max, 0.001)
x_cont = np.sin(2 * np.pi * f * t_cont)

# 2. Definindo as três frequências de amostragem diferentes
fs_list = [100.0, 25.0, 8.0]

# Configurando a figura
plt.figure(figsize=(10, 8))

# Loop para processar e plotar cada frequência de amostragem
for i, fs in enumerate(fs_list):
    Ts = 1.0 / fs

    # Instantes de amostragem (tn = n * Ts)
    n = np.arange(0, int(t_max * fs))
    t_disc = n * Ts

    # Amostrando o sinal: x[n] = x(n * Ts)
    x_disc = np.sin(2 * np.pi * f * t_disc)

    # Criando os subplots
    plt.subplot(3, 1, i + 1)

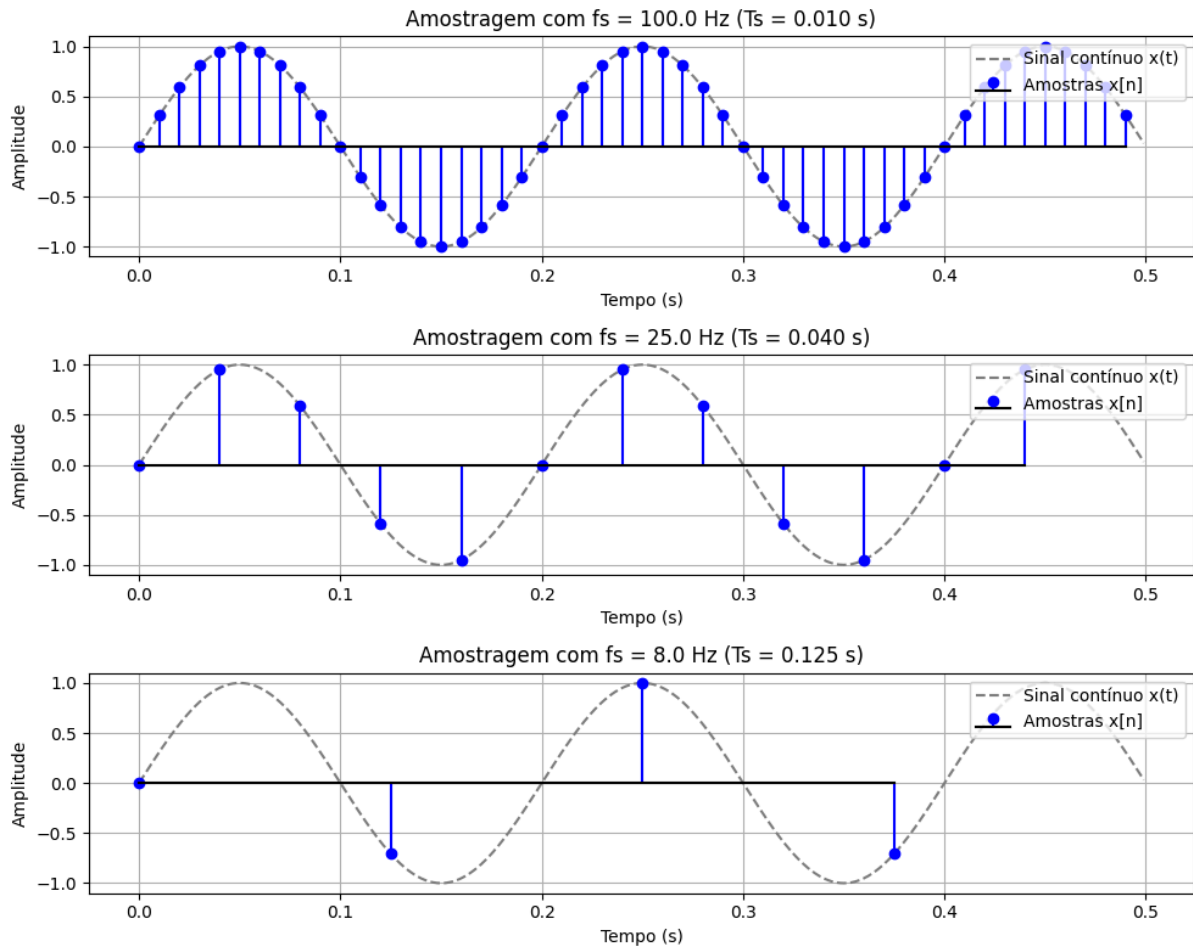
    # Plotando o sinal contínuo (linha de fundo)
    plt.plot(t_cont, x_cont, color='gray', linestyle='--', label='Sinal contínuo x(t)')

    # Plotando as amostras discretas por cima (stem plot)
    plt.stem(t_disc, x_disc, linefmt='b-', markerfmt='bo', basefmt='k-', label=f'Amostras x[n]')

    plt.title(f'Amostragem com fs = {fs} Hz (Ts = {Ts:.3f} s)')
    plt.xlabel('Tempo (s)')
    plt.ylabel('Amplitude')
    plt.legend(loc='upper right')
    plt.grid(True)

plt.tight_layout()
plt.show()

```



2.5. Resultados

Ao rodar a simulação, você obterá uma figura com três gráficos empilhados, todos mostrando o mesmo sinal senoidal tracejado de fundo (o sinal real e contínuo), mas com marcações (hastes azuis) diferentes representando as amostras coletadas:

1. Gráfico 1 ($f_s = 100$ Hz): As hastes azuis estão muito próximas umas das outras. O contorno formado pelas pontas das amostras desenha perfeitamente a onda senoidal de fundo.
2. Gráfico 2 ($f_s = 25$ Hz): O espaçamento entre as hastes é maior. Ainda é possível ver o formato de "onda" subindo e descendo visualmente, mas já temos bem menos pontos de referência por cada ciclo da senoide (apenas 5 pontos por ciclo).
3. Gráfico 3 ($f_s = 8$ Hz): O espaçamento é enorme. Os pontos parecem estar quase espalhados aleatoriamente se você tentar ligá-los mentalmente. Visualmente, a sequência de amostras não se parece em nada com a senoide bonita e suave que está no fundo.

2.6. Discussão

A comparação dos resultados ilustra de forma muito clara como a frequência de amostragem (f_s) modifica a representação discreta do sinal.

Matematicamente, a operação $x[n] = x(n * T_s)$ "apaga" toda a informação contida no sinal original entre um instante t_n e o próximo. Sendo assim, se a frequência de amostragem for muito alta (como em $f_s = 100$ Hz), o período T_s é minúsculo. A consequência é que as amostras capturam com alta fidelidade as variações de subida e descida da onda original.

No entanto, à medida que diminuirmos a frequência de amostragem (aumentando o período T_s , como no caso de 8 Hz), nós passamos a ficar "cegos" por mais tempo entre uma foto e outra. A representação discreta passa a ser uma versão muito pobre do fenômeno físico. Se entregarmos apenas os pontos azuis do terceiro gráfico para um computador, ele jamais conseguiria deduzir visualmente que aquilo era a senoide de 5 Hz desenhada no fundo.

Isso evidencia que, para representar um sinal corretamente no mundo digital, não basta apenas amostrá-lo; é necessário escolher uma frequência de amostragem rápida o suficiente para capturar as flutuações mais rápidas daquele sinal (conceito que fundamentará o estudo do Teorema de Nyquist mais à frente).

Aqui está o material estruturado para o **Tópico 3 – Quantização e Resolução**, baseado na imagem "image_e5f1bb.png". Mantive o formato de texto simples para as equações matemáticas, garantindo que você consiga copiar e colar diretamente no Google Docs sem nenhum bug de formatação.

3.1. Quantização e Resolução

Enquanto a amostragem trata de "quebrar" o eixo do tempo em instantes discretos, a quantização trata de "quebrar" o eixo da amplitude.

No mundo físico, uma grandeza como a tensão elétrica pode assumir infinitos valores contínuos (por exemplo, 2.71828... V). No entanto, sistemas digitais como microcontroladores e computadores possuem uma memória finita baseada em bits (0s e 1s). Portanto, é impossível armazenar um número com infinitas casas decimais.

A quantização é o processo de mapear essa infinidade de valores contínuos para um conjunto finito e restrito de níveis discretos pré-estabelecidos. Ao fazermos isso, o valor real do sinal precisa ser arredondado para o nível permitido mais próximo. Esse arredondamento gera uma pequena perda de informação, que nós chamamos de erro de quantização (ou ruído de quantização).

3.2. Formulação matemática

O desempenho de um processo de quantização depende da quantidade de bits do conversor. As principais formulações são:

- Quantidade de níveis (L): É o número total de "degraus" ou valores de amplitude distintos que o sistema digital pode representar.
- $L = 2^N$

- Onde N é o número de bits de resolução do sistema.
- Resolução ou Passo de Quantização (ΔV): É a menor diferença de tensão que o sistema consegue detectar e representar. Representa o "tamanho" ou a altura de cada degrau. Considerando que o conversor opera em uma faixa que vai de 0 V até uma tensão de referência máxima Vref:
- $\Delta V = V_{ref} / 2^N$
- Erro de quantização (e): É a diferença matemática entre o sinal real contínuo e o sinal já quantizado em degraus.
- $e[n] = \text{sinal_quantizado}[n] - \text{sinal_original}[n]$

3.3. Exemplo

Vamos imaginar um sensor de temperatura que envia para um microcontrolador uma tensão analógica que varia de 0 a 5 Volts ($V_{ref} = 5 \text{ V}$). O conversor do microcontrolador possui apenas 3 bits de resolução ($N = 3$).

- Número de níveis: $L = 2^3 = 8$ níveis possíveis (representados em binário de 000 a 111).
- Resolução: $\Delta V = 5 / 8 = 0.625 \text{ V}$.

Isso significa que o sistema digital só consegue ler tensões em múltiplos de 0.625 V (0 V, 0.625 V, 1.25 V, 1.875 V, etc.). Se o sensor enviar exatamente 1.0 V (sinal original), o sistema será forçado a arredondar esse valor para o degrau mais próximo. Se a lógica do conversor for de "arredondar para baixo" (truncamento), ele registrará 0.625 V. O erro de quantização nesse instante será de $0.625 - 1.0 = -0.375 \text{ V}$.

3.4. Simulação

O script Python a seguir gera um sinal senoidal que varia de 0 a 5V e simula o processo de quantização para $N = 3$, $N = 4$ e $N = 8$ bits, calculando os níveis, a resolução e o erro, além de plotar os gráficos lado a lado.

```
import numpy as np
import matplotlib.pyplot as plt

# 1. Parâmetros do Sinal Original
t = np.arange(0, 1, 0.001) # Tempo de 0 a 1 segundo
f = 1.0                     # Frequência de 1 Hz
Vref = 5.0                  # Tensão de referência (fundo de escala)

# Gerando um sinal que varia entre 0V e 5V (Senoide deslocada e escalonada)
sinal_original = (Vref / 2) + (Vref / 2) * np.sin(2 * np.pi * f * t)

# Casos de estudo solicitados: 3, 4 e 8 bits
```

```

bits_N = [3, 4, 8]

plt.figure(figsize=(14, 10))

for i, N in enumerate(bits_N):
    # Cálculos matemáticos
    L = 2**N
    delta_V = Vref / L

    # 2. Processo de Quantização
    # Divide a tensão pelo passo ( $\Delta V$ ) e pega a parte inteira (índice do
    degrau)
    indice_degrau = np.floor(sinal_original / delta_V)
    # Garante que o índice não passe do máximo permitido (L-1)
    indice_degrau = np.clip(indice_degrau, 0, L - 1)
    # Multiplica pelo passo para obter a tensão quantizada final
    sinal_quantizado = indice_degrau * delta_V

    # 3. Cálculo do erro
    erro_quantizacao = sinal_quantizado - sinal_original

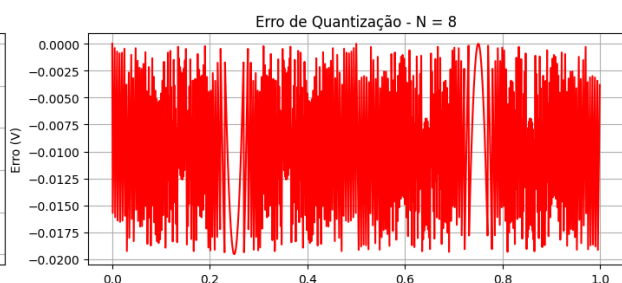
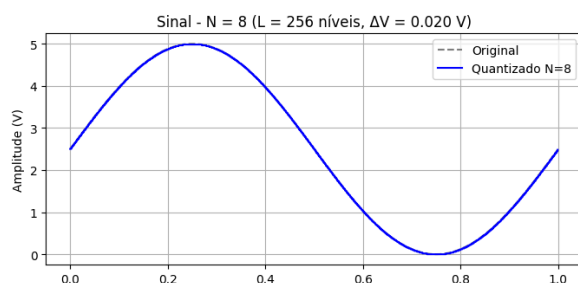
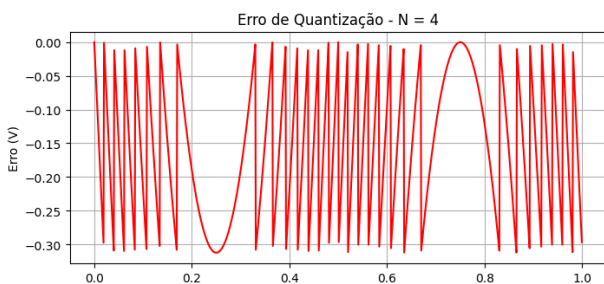
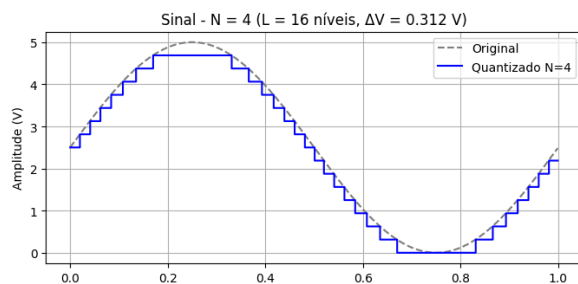
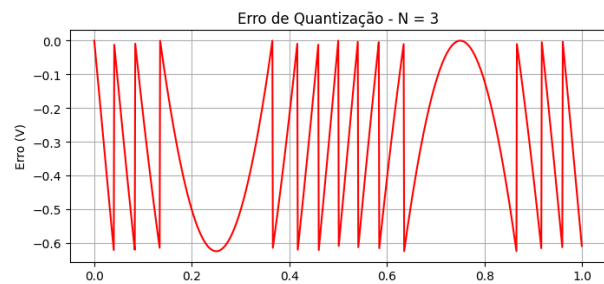
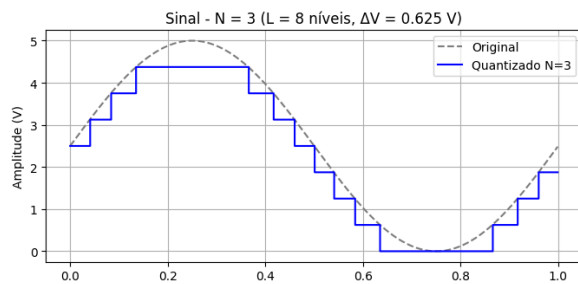
    # --- Gráficos do Sinal ---
    plt.subplot(3, 2, 2*i + 1)
    plt.plot(t, sinal_original, 'gray', linestyle='--',
label='Original')
    plt.step(t, sinal_quantizado, 'b', where='post', label=f'Quantizado
N={N}')
    plt.title(f'Sinal - N = {N} (L = {L} níveis,  $\Delta V = \{delta\_V:.3f\}$ 
V)')
    plt.ylabel('Amplitude (V)')
    plt.legend(loc='upper right')
    plt.grid(True)

    # --- Gráficos do Erro ---
    plt.subplot(3, 2, 2*i + 2)
    plt.plot(t, erro_quantizacao, 'r')
    plt.title(f'Erro de Quantização - N = {N}')
    plt.ylabel('Erro (V)')
    plt.grid(True)

plt.tight_layout()
plt.show()

```

```
# Imprimindo tabela de resultados numéricos no terminal
print(" N (bits) | Níveis (L) | Resolução ( $\Delta V$ ) em V | Erro Máximo (aprox) em V")
print("-" * 75)
for N in bits_N:
    L = 2**N
    delta_V = Vref / L
    print(f"      {N}          |      {L:3}          |      {delta_V:.5f}          |
± {delta_V:.5f}")
```



3.5. Resultados

A execução da simulação gera uma janela com 6 gráficos (3 linhas e 2 colunas). A coluna da esquerda mostra os sinais sobrepostos, e a da direita mostra o erro isolado. A saída no terminal será:

N (bits)	Níveis (L)	Resolução (ΔV) em V	Erro Máximo (aprox) em V
3	8	0.62500	± 0.62500

4	16	0.31250	± 0.31250
8	256	0.01953	± 0.01953

Visualmente, você observará:

1. Caso $N = 3$: O sinal quantizado parece uma "escadaria" bem grossa, com degraus enormes. A onda original cinza fica bem distorcida e distante dos degraus azuis. O gráfico do erro mostra picos grandes, alcançando até 0.625 V em módulo.
2. Caso $N = 4$: O formato de escada ainda é visível, mas agora com o dobro de degraus. O sinal azul acompanha muito melhor as curvas do sinal cinza. O erro foi reduzido pela metade.
3. Caso $N = 8$: Visualmente, a "escadaria" azul fica tão fina que se sobrepõe quase perfeitamente à onda cinza. A olho nu, parece um sinal contínuo. O gráfico do erro na direita é quase uma linha reta perto de zero (oscilando em limites muito minúsculos de 0.019 V).

3.6. Discussão

A análise visual e numérica evidencia uma relação direta e fundamental em sistemas digitais: quanto maior o número de bits (N), exponencialmente maior será a quantidade de níveis disponíveis ($L = 2^N$).

Ter mais níveis significa que o nosso "degrau" de aproximação (resolução ΔV) fica cada vez menor. Como o conversor tem dezenas ou centenas de opções de valores a mais para escolher na hora de arredondar, o valor escolhido ficará extremamente próximo do valor físico real.

É exatamente por isso que o erro de quantização diminui substancialmente quando aumentamos a resolução. No caso extremo da nossa simulação ($N=8$ bits), temos 256 níveis. A diferença máxima que um sinal sofrerá ao ser digitalizado será de, no máximo, o tamanho de um degrau (cerca de 0.019 V), o que para grande parte das aplicações de controle e medição generalista já é um erro completamente desprezível.

Na engenharia, a escolha de N é sempre um compromisso (trade-off) de projeto: usar 8, 12, 16 ou 24 bits torna a representação incrivelmente fiel à realidade (baixo erro), mas exige componentes analógico-digitais (ADCs) mais caros e consome muito mais espaço na memória e processamento do sistema para lidar com números binários maiores.

4.1. Sequências Fundamentais

No estudo de Processamento Digital de Sinais (PDS), as sequências fundamentais servem como os blocos de construção básicos (building blocks) para a análise e síntese de qualquer sinal discreto complexo e para a caracterização de Sistemas Lineares e Invariantes no Tempo (LTI).

- Impulso Unitário ($\delta[n]$): É o equivalente discreto da função delta de Dirac. Representa uma perturbação pontual que ocorre exclusivamente no índice $n = 0$. Funciona como o "átomo" dos sinais discretos: qualquer sinal arbitrário pode ser decomposto como uma combinação linear de impulsos deslocados e escalonados.
- Degrau Unitário ($u[n]$): Representa um sinal de chaveamento que muda de estado (de 0 para 1) no instante $n = 0$ e assim permanece para todo $n \geq 0$. É fundamental para modelar sistemas causais (que iniciam sua operação a partir de um instante zero).
- Sequência Exponencial (a^n): Modela processos de amortecimento, crescimento ou oscilação. O parâmetro " a " dita o comportamento dinâmico do sinal no tempo.
- Senoide Discreta ($A \cdot \cos(w_0 \cdot n + \phi)$) e Exponencial Complexa ($e^{j \cdot w_0 \cdot n}$): Representam os sinais oscilatórios e periódicos no domínio discreto. Pela Identidade de Euler, as exponenciais complexas e as senoides estão intimamente ligadas: uma exponencial complexa sintetiza, simultaneamente, uma parte real cosenoidal e uma parte imaginária senoidal. Essa relação é a pedra angular da Transformada de Fourier.

4.2. Formulação matemática

- Impulso Unitário:
- $\delta[n] = 1$, para $n = 0$
- $\delta[n] = 0$, para $n \neq 0$

Propriedade da Amostragem / Decomposição em Impulsos:

$$x[n] = \text{soma de } k = -\infty \text{ a } +\infty \text{ de } \{ x[k] \cdot \delta[n - k] \}$$

Significado: Esta propriedade estabelece que qualquer sequência $x[n]$ pode ser vista como uma soma infinita de impulsos deslocados para a posição k , onde cada impulso $\delta[n - k]$ é multiplicado (pesado) pela amplitude do sinal $x[k]$ naquele exato ponto k .

- Degrau Unitário:
- $u[n] = 1$, para $n \geq 0$
- $u[n] = 0$, para $n < 0$

Relação com o Impulso:

$$\delta[n] = u[n] - u[n - 1]$$

Significado: O impulso unitário no instante $n = 0$ é obtido pela diferença entre o degrau atual e o degrau atrasado de uma amostra. Analogamente, o degrau é o acumulador (soma corrida) do impulso: $u[n] = \text{soma de } k = -\infty \text{ até } n \text{ de } \{\delta[k]\}$.

- Exponencial Discreta:
- Dada por $x[n] = a^n$, onde "a" pode ser real ou complexo. Analisando "a" pertencente aos Reais:
- Se $|a| > 1$: Sequência exponencialmente crescente.
- Se $0 < a < 1$: Sequência exponencialmente decrescente (decaimento suave).
- Se $-1 < a < 0$: Sequência exponencialmente decrescente com alternância de sinal (oscila entre valores positivos e negativos).
- Se $a < -1$: Sequência exponencialmente crescente com alternância de sinal.
- Senoide Discreta e Exponencial Complexa:
- A senoide discreta é definida por:
- $x[n] = A * \cos(w_0 * n + \phi)$

Onde A é a Amplitude, w_0 é a frequência angular discreta em rad/amostra e ϕ é a fase inicial em radianos.

Pela Identidade de Euler:

$$e^{j * \theta} = \cos(\theta) + j * \sin(\theta)$$

Substituindo θ por $w_0 * n$:

$$e^{j * w_0 * n} = \cos(w_0 * n) + j * \sin(w_0 * n)$$

Reorganizando, podemos expressar as funções trigonométricas puras como combinações de exponenciais complexas:

$$\cos(w_0 * n) = (e^{j * w_0 * n} + e^{-j * w_0 * n}) / 2$$

$$\sin(w_0 * n) = (e^{j * w_0 * n} - e^{-j * w_0 * n}) / (2 * j)$$

4.3. Exemplo

Para fins de simulação e análise, vamos definir os seguintes parâmetros para cada uma das quatro sequências em um intervalo de índices n entre -5 e 15:

1. Impulso Unitário: $\delta[n]$
2. Degrau Unitário: $u[n]$
3. Exponencial Real Causativa: $x_3[n] = (a^n) * u[n]$, escolhendo $a = 0.8$ (decaimento exponencial suave para $n \geq 0$).

4. Senoide Discreta: $x_4[n] = A * \cos(w_0 * n + \phi)$, escolhendo $A = 2.0$, $w_0 = \pi/4$ rad/amostra (frequência que completa 1 ciclo a cada 8 amostras) e $\phi = 0$.

4.4. Simulação

O código Python a seguir utiliza NumPy e Matplotlib para gerar e plotar as quatro sequências discretas fundamentais sem formatação especial que piore a cópia.

```
import numpy as np
import matplotlib.pyplot as plt

# Definindo o intervalo do índice discreto n
n = np.arange(-5, 16)

# 1. Impulso Unitário delta[n]
delta = np.where(n == 0, 1.0, 0.0)

# 2. Degrau Unitário u[n]
u = np.where(n >= 0, 1.0, 0.0)

# 3. Exponencial Real Causativa  $a^n * u[n]$  com  $a = 0.8$ 
a = 0.8
exp_seq = (a**n) * u

# 4. Senoide Discreta  $A * \cos(w_0 * n + \phi)$ 
A = 2.0
w0 = np.pi / 4 # 0.25 * pi rad/amostra (Período N = 8 amostras)
phi = 0.0
senoide = A * np.cos(w0 * n + phi)

# --- Plotagem das Sequências ---
plt.figure(figsize=(12, 10))

# Subplot 1: Impulso Unitário
plt.subplot(4, 1, 1)
plt.stem(n, delta, linefmt='b-', markerfmt='bo', basefmt='k-')
plt.title('1. Impulso Unitario: delta[n]')
plt.ylabel('Amplitude')
plt.grid(True)

# Subplot 2: Degrau Unitário
plt.subplot(4, 1, 2)
plt.stem(n, u, linefmt='g-', markerfmt='go', basefmt='k-')
plt.title('2. Degrau Unitario: u[n]')
```

```

plt.ylabel('Amplitude')
plt.grid(True)

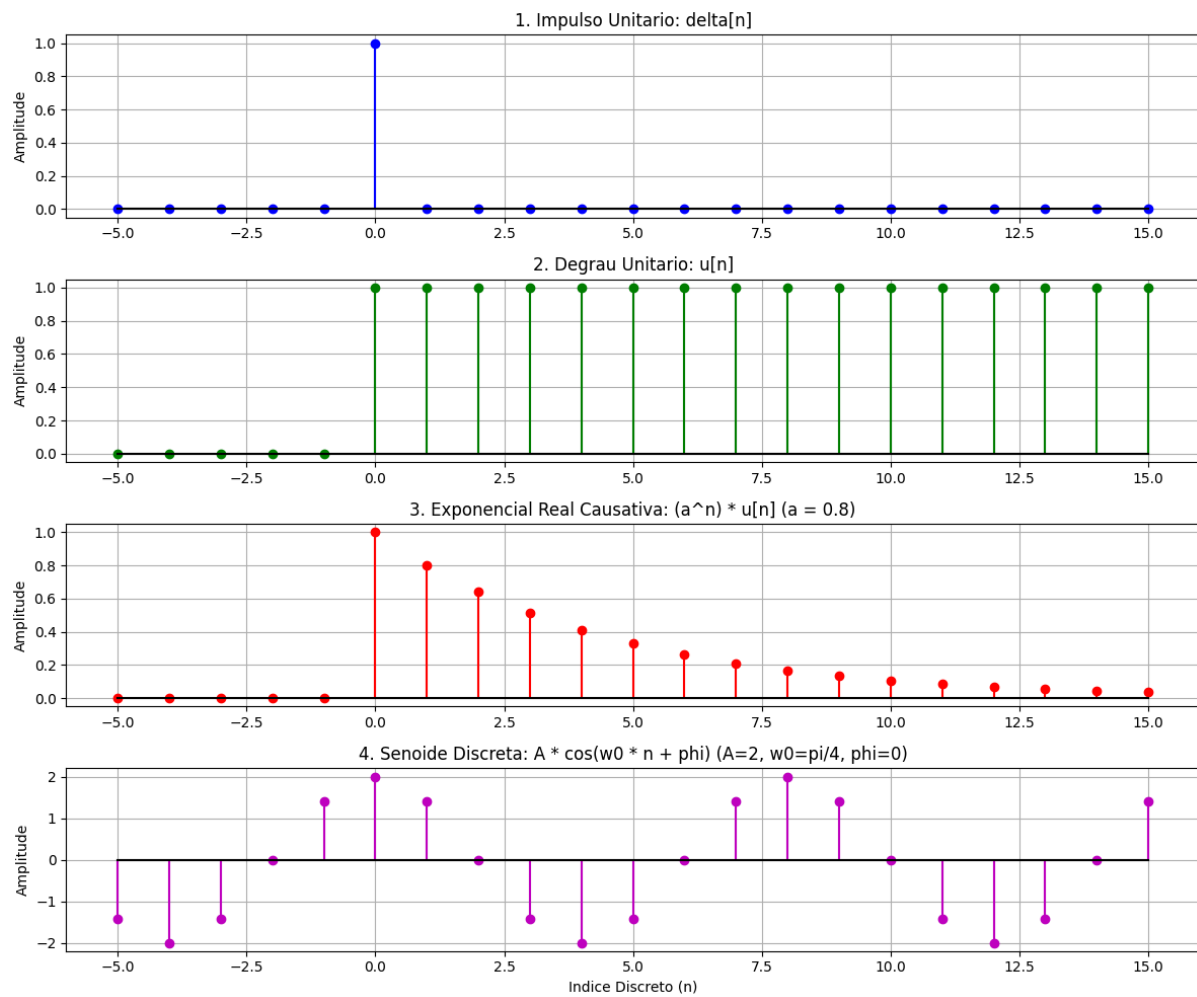
# Subplot 3: Exponencial Real Causativa
plt.subplot(4, 1, 3)
plt.stem(n, exp_seq, linefmt='r-', markerfmt='ro', basefmt='k-')
plt.title(f'3. Exponencial Real Causativa:  $(a^n) * u[n]$  ( $a = \{a\}$ )')
plt.ylabel('Amplitude')
plt.grid(True)

# Subplot 4: Senoide Discreta
plt.subplot(4, 1, 4)
plt.stem(n, senoide, linefmt='m-', markerfmt='mo', basefmt='k-')
plt.title('4. Senoide Discreta:  $A * \cos(w_0 * n + \phi)$  ( $A=2$ ,  $w_0=\pi/4$ ,  $\phi=0$ )')
plt.xlabel('Indice Discreto (n)')
plt.ylabel('Amplitude')
plt.grid(True)

plt.tight_layout()
plt.show()

# Tabela numérica demonstrativa para os primeiros índices
print(" n | delta[n] | u[n] |  $(0.8)^n * u[n]$  |  $2*\cos(\pi/4 * n)$ ")
print("-" * 55)
for i in range(5, 11): # amostragem de n = 0 até n = 5
    print(f"{n[i]:2d} | {delta[i]:.1f} | {u[i]:.1f} | {exp_seq[i]:.4f} | {senoide[i]:.4f}")

```



4.5. Resultados

A tabela a seguir apresenta os valores numéricos gerados pelo script para os índices de $n = 0$ a $n = 5$:

n	$\delta[n]$	$u[n]$	$(0.8)^n * u[n]$	$2 * \cos(\pi/4 * n)$
0	1.0	1.0	1.0000	2.0000
1	0.0	1.0	0.8000	1.4142
2	0.0	1.0	0.6400	0.0000

3	0.0	1.0	0.5120	-1.4142
4	0.0	1.0	0.4096	-2.0000
5	0.0	1.0	0.3277	-1.4142

Visualmente, a execução do código produz quatro gráficos de hastes (stem plots):

1. $\delta[n]$: Apresenta um único pulso vertical de valor 1 em $n = 0$, permanecendo zerado em todos os outros índices (negativos e positivos).
2. $u[n]$: Apresenta valor 0 para todo $n < 0$ e uma sequência constante de hastes de altura 1 para $n \geq 0$.
3. $(0.8^n) * u[n]$: Zerado para $n < 0$. Começa em 1.0 em $n = 0$ e decresce gradualmente rumo a zero à medida que n avança.
4. $2 * \cos(\pi/4 * n)$: Uma oscilação discreta que varia entre +2.0 e -2.0, completando um ciclo exato a cada $N = (2 * \pi) / \omega_0 = (2 * \pi) / (\pi / 4) = 8$ amostras.

4.6. Discussão

A análise gráfica e matemática das sequências discretas fundamentadas neste tópico traz conclusões centrais para a teoria de processamento de sinais:

- Significado da Decomposição por Impulso: A propriedade $x[n] = \text{soma } \{ x[k] * \delta[n - k] \}$ mostra que a função impulso funciona como a "base ortogonal" padrão do domínio do tempo discreto. Em termos de engenharia, se soubermos a resposta de um sistema linear e invariante no tempo (LTI) a um único impulso $\delta[n]$ — conhecida como a Resposta ao Impulso $h[n]$ —, seremos capazes de calcular a resposta do sistema para qualquer sinal de entrada $x[n]$ por meio da operação de convolução discreta.
- Interdependência entre $u[n]$ e $\delta[n]$: A relação $\delta[n] = u[n] - u[n-1]$ demonstra como sinais descontínuos podem ser obtidos a partir de operações simples de atraso e subtração. A multiplicação de qualquer sequência por $u[n]$ impõe a condição de causalidade, zerando os valores para instantes do passado ($n < 0$).
- Comportamento da Exponencial a^n : A escolha de $a = 0.8$ gerou uma sequência estável (BIBO - bounded-input, bounded-output), pois a soma de seus valores converge. Se tivéssemos escolhido $a = -0.8$, os valores alternariam de sinal (+1.0, -0.8, +0.64, -0.512...). Se $|a| > 1$, o sinal seria divergente/instável.

- **Conexão de Euler e Transformada de Fourier:** A decomposição da exponencial complexa $e^{(j * w_0 * n)}$ em senos e cossenos reais demonstra que exponenciais complexas são as "funções próprias" (eigenfunctions) dos sistemas LTI. Quando aplicamos $e^{(j * w_0 * n)}$ em um sistema LTI, a saída é a mesma exponencial complexa apenas multiplicada por um número complexo (ganho e fase do sistema). Essa propriedade simplifica equações diferenciais/de diferenças em equações algébricas no domínio da frequência, formando a base teórico-prática da Transformada de Fourier Discreta (DFT).

5.1. Operações com Sinais

No processamento digital de sinais, as operações fundamentais sobre sequências discretas $x[n]$ dividem-se em duas categorias principais: transformações na variável independente (tempo n) e transformações na variável dependente (amplitude do sinal).

- **Operações no Domínio do Tempo (Variável Independente n):**
 - **Deslocamento Temporal:** Altera o instante em que a informação ocorre. Quando atrasamos um sinal, ele é movido para o futuro (direita do gráfico); quando avançamos, ele é movido para o passado (esquerda do gráfico).
 - **Inversão Temporal:** Espelha o sinal em relação ao eixo vertical ($n = 0$). O que acontecia em instantes futuros passa a acontecer em instantes passados com o mesmo valor de amplitude.
- **Operações no Domínio da Amplitude (Variável Dependente):**
 - **Escalonamento de Amplitude:** Altera a magnitude do sinal multiplicando-o por uma constante A . Se $|A| > 1$, ocorre uma amplificação; se $0 < |A| < 1$, ocorre uma atenuação. Se A for negativo, o sinal também sofre uma inversão de polaridade (ou fase).
 - **Soma de Sinais:** Combina dois ou mais sinais somando suas amplitudes ponto a ponto para cada índice n idêntico. É a base do princípio da superposição em sistemas lineares.

5.2. Formulação matemática

Considerando uma sequência discreta original $x[n]$:

- **Deslocamento Temporal:** $y[n] = x[n - n_0]$
 - Se $n_0 > 0$: Atraso de n_0 amostras (deslocamento para a direita).
 - Se $n_0 < 0$: Avanço de $|n_0|$ amostras (deslocamento para a esquerda).
- **Inversão Temporal:** $y[n] = x[-n]$

- O valor da amostra no índice k é atribuído ao índice $-k$.
- **Escalonamento de Amplitude:** $y[n] = A * x[n]$
- Onde A é um número real (ganho do sistema).
- **Soma de Sinais:** $y[n] = x1[n] + x2[n]$
- Para cada índice n , calcula-se a soma aritmética direta dos valores de $x1[n]$ e $x2[n]$.

5.3. Exemplo

Para demonstrar as transformações sem ambiguidade, escolheremos uma sequência $x[n]$ assimétrica definida no intervalo n entre -5 e 5:

$x[n] = n$, para $0 \leq n \leq 3$, e $x[n] = 0$ para qualquer outro valor de n .

Valores do sinal original $x[n]$:

- $x[0] = 0$
- $x[1] = 1$
- $x[2] = 2$
- $x[3] = 3$
- $x[n] = 0$ para $n < 0$ e $n > 3$.

Aplicando as 5 operações solicitadas:

1. **Atraso ($x[n - 2]$):** A amostra em $n=0$ vai para $n=2$. O sinal existirá entre $n = 2$ e $n = 5$.
2. **Avanço ($x[n + 2]$):** A amostra em $n=0$ vai para $n=-2$. O sinal existirá entre $n = -2$ e $n = 1$.
3. **Inversão ($x[-n]$):** A amostra em $n=3$ vai para $n=-3$, $n=2$ vai para $n=-2$, $n=1$ vai para $n=-1$.
4. **Amplificação ($2 * x[n]$):** Amplitudes multiplicadas por 2 (valores passam a ser 0, 2, 4, 6 nos mesmos índices 0, 1, 2, 3).
5. **Inversão de Polaridade ($-x[n]$):** Amplitudes multiplicadas por -1 (valores passam a ser 0, -1, -2, -3 nos mesmos índices 0, 1, 2, 3).

5.4. Simulação

O script em Python a seguir gera o sinal original e plota em uma grade de gráficos comparativos todas as transformações operadas sobre o tempo e a amplitude.

```

import numpy as np
import matplotlib.pyplot as plt

# Definindo o vetor de índices n
n = np.arange(-6, 7)

# Definindo o sinal original x[n] (Rampa assimétrica de 4 pontos)
# x[n] = n para 0 ≤ n ≤ 3, senão 0
x = np.where((n ≥ 0) & (n ≤ 3), n, 0)

# Operações
x_atrasado = np.where(((n - 2) ≥ 0) & ((n - 2) ≤ 3), n - 2, 0) # x[n
- 2]
x_avancado = np.where(((n + 2) ≥ 0) & ((n + 2) ≤ 3), n + 2, 0) # x[n
+ 2]
x_invertido = np.where((-n) ≥ 0 & ((-n) ≤ 3), -n, 0) #
x[-n]
x_amplificado = 2 * x # 2 *
x[n]
x_polaridade = -x #
-x[n]

# Plotagem dos resultados
plt.figure(figsize=(14, 10))

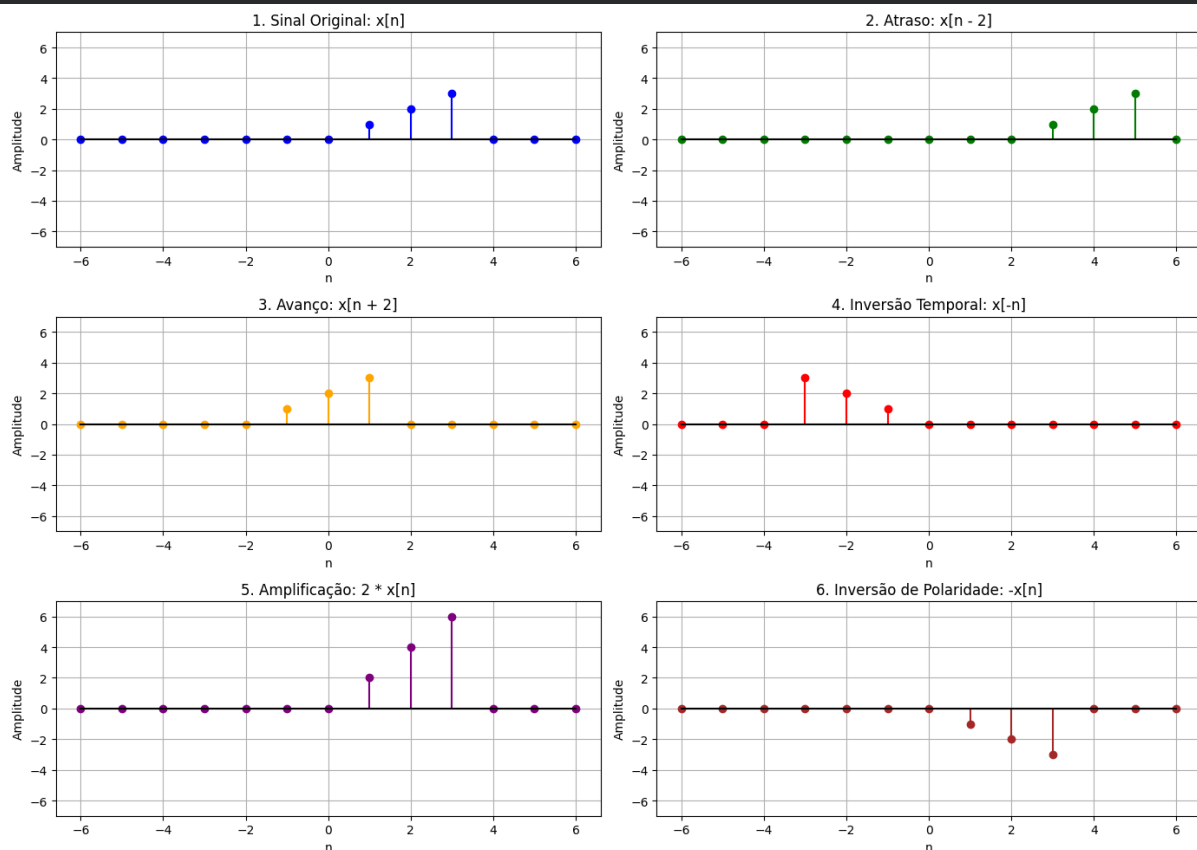
sinais = [
    (x, '1. Sinal Original: x[n]', 'blue'),
    (x_atrasado, '2. Atraso: x[n - 2]', 'green'),
    (x_avancado, '3. Avanço: x[n + 2]', 'orange'),
    (x_invertido, '4. Inversão Temporal: x[-n]', 'red'),
    (x_amplificado, '5. Amplificação: 2 * x[n]', 'purple'),
    (x_polaridade, '6. Inversão de Polaridade: -x[n]', 'brown')
]

for i, (sig, titulo, cor) in enumerate(sinais, 1):
    plt.subplot(3, 2, i)
    plt.stem(n, sig, linefmt=f'{cor}', markerfmt=f'{cor}', basefmt='k')
    plt.title(titulo)
    plt.xlabel('n')
    plt.ylabel('Amplitude')
    plt.ylim(-7, 7)
    plt.grid(True)

```

```
plt.tight_layout()
plt.show()

# Impressão numérica comparativa
print(" n | x[n] | x[n-2] | x[n+2] | x[-n] | 2*x[n] | -x[n]")
print("-" * 52)
for idx in range(len(n)):
    print(f"{n[idx]:2d} | {x[idx]:.0f} | {x_atrasado[idx]:.0f} | {x_avancado[idx]:.0f} | {x_invertido[idx]:.0f} | {x_amplificado[idx]:.0f} | {x_polaridade[idx]:.0f}")
```



5.5. Resultados

A tabela abaixo exibe os valores numéricos obtidos no terminal para a faixa de índices n entre -3 e 5:

n	x[n]	x[n-2]	x[n+2]	x[-n]	2*x[n]	-x[n]
-3	0	0	0	3	0	0

-2	0	0	0	2	0	0
-1	0	0	1	1	0	0
0	0	0	2	0	0	0
1	1	0	3	0	2	-1
2	2	0	0	0	4	-2
3	3	1	0	0	6	-3
4	0	2	0	0	0	0
5	0	3	0	0	0	0

Visualmente nos gráficos gerados:

1. **$x[n]$** : Rampa crescente de 0 a 3 localizada nos índices 0, 1, 2, 3.
2. **$x[n - 2]$** : A mesma forma de onda deslocada duas posições para a direita (ocupando $n = 2, 3, 4, 5$).
3. **$x[n + 2]$** : A mesma forma de onda deslocada duas posições para a esquerda (ocupando $n = -2, -1, 0, 1$).
4. **$x[-n]$** : A rampa aparece espelhada no eixo negativo (ocupando $n = -3, -2, -1, 0$ com valores 3, 2, 1, 0 respectivamente).
5. **$2 * x[n]$** : Mantém a posição temporal original ($n = 0$ a 3), mas os picos de amplitude sobem até 6.
6. **$-x[n]$** : Mantém a posição temporal original ($n = 0$ a 3), mas as hastes apontam para baixo, atingindo -3 no ponto máximo.

5.6. Discussão

A análise matemática e visual dos resultados confirma o comportamento de cada operação sobre as variáveis do sinal:

- **Efeito das Transformações no Tempo ($x[n - n_0]$ e $x[n + n_0]$):** Na operação $x[n - 2]$, para encontrar o valor do novo sinal em $n = 2$, o sistema busca a amostra original em $x[2 - 2] = x[0]$. Isso faz com que todo o bloco de dados seja reproduzido com um retardo de 2 instantes temporais. O inverso ocorre em $x[n + 2]$, onde para $n = -2$ o sistema acessa $x[-2 + 2] = x[0]$, adiantando o aparecimento do sinal.
- **Efeito da Inversão Temporal ($x[-n]$):** Ao substituir a variável n por $-n$, o sentido de leitura do tempo é revertido. A amostra mais distante no futuro ($x[3] = 3$) torna-se a amostra mais distante no passado ($x[-3] = 3$). Essa operação é fundamental em PDS para o cálculo do algoritmo de **convolução discreta**, onde um dos sinais precisa ser invertido no tempo antes de ser deslocado e multiplicado.
- **Efeito do Escalonamento de Amplitude ($A * x[n]$):** A operação $2 * x[n]$ multiplica cada amostra individualmente por um ganho constante de 2, mantendo a variável independente n intocada. O tempo não sofre dilatação ou deslocamento. Quando o fator de ganho é negativo (-1 em $-x[n]$), a forma do sinal é refletida horizontalmente em relação ao eixo do tempo, alterando sua fase em 180 graus sem modificar sua duração ou posição temporal.

6.1. Energia e potência

No contexto de sinais e sistemas, os termos "energia" e "potência" não se referem necessariamente às grandezas físicas medidas em Joules e Watts (a menos que o sinal represente exatamente tensão ou corrente sobre um resistor de 1 Ohm). Na matemática de sinais, eles são métricas que quantificam o "tamanho" ou a "força" geral de uma sequência.

- **Sinais de Energia:** São sinais que possuem uma energia total finita (maior que zero e menor que infinito). Como a energia é finita sobre um tempo infinito, a potência média desse sinal distribuída ao longo de todo o tempo é inevitavelmente zero. Geralmente, sinais que têm duração finita (como um pulso rápido) ou que decaem para zero com o passar do tempo (como uma exponencial decrescente) são sinais de energia.
- **Sinais de Potência:** São sinais que não decaem para zero, existindo indefinidamente no tempo (como uma onda senoidal contínua, um ruído constante ou um sinal de corrente contínua - DC). Se tentarmos somar a energia desse sinal de menos a mais infinito, o resultado será infinito. No entanto, se calcularmos a média dessa energia por amostra, obteremos um valor finito e não nulo. Isso os classifica como sinais de potência.

Essas duas classificações são mutuamente exclusivas: um sinal com energia finita tem potência zero, e um sinal com potência finita (diferente de zero) tem energia infinita.

6.2. Formulação matemática

- Energia de um sinal discreto (E):
- É a soma dos quadrados dos módulos (amplitudes) de todas as amostras do sinal, de menos infinito a mais infinito.
- $E = \text{soma, de } n = -\text{infinito} \text{ até } +\text{infinito, de } |x[n]|^2$
-

Um sinal é classificado como "sinal de energia" quando: $0 < E < \text{infinito}$.

- Potência média de um sinal discreto (P):
- É a média da energia ao longo de um intervalo de observação, quando esse intervalo tende ao infinito. O termo $(2N + 1)$ representa a quantidade total de amostras no intervalo fechado de $-N$ até N .
- $P = \lim_{N \rightarrow \infty} \left[\frac{1}{(2N + 1)} * \text{soma, de } n = -N \text{ até } N, \text{ de } |x[n]|^2 \right]$
-

Um sinal é classificado como "sinal de potência" quando: $0 < P < \text{infinito}$.

6.3. Exemplo

Vamos selecionar um exemplo clássico para cada tipo e resolvê-los analiticamente:

- Exemplo 1: Sinal de Energia (Pulso Retangular):
 - Seja $x_1[n] = 2$ para $0 \leq n \leq 4$ (5 amostras) e $x_1[n] = 0$ para outros valores.
- Cálculo analítico da Energia:
 - $E = |2|^2 + |2|^2 + |2|^2 + |2|^2 + |2|^2$
 - $E = 4 + 4 + 4 + 4 + 4 = 20$. (Energia finita!)
 - Como a duração é finita, ao dividir 20 por um período infinito, a Potência é 0.
 - Exemplo 2: Sinal de Potência (Senoide Discreta Infinita)
- Seja $x_2[n] = 2 * \sin(\pi * n / 2)$ para todo n de menos a mais infinito.
- Se calcularmos a sequência para $n = 0, 1, 2, 3, \dots$ teremos os valores: 0, 2, 0, -2, 0, 2, 0, -2, ...
- A energia em um período (4 amostras) é: $0^2 + 2^2 + 0^2 + (-2)^2 = 0 + 4 + 0 + 4 = 8$.
- Como ela se repete infinitamente, a Energia total é infinita.
- Cálculo analítico da Potência (média sobre 1 período):
- $P = \text{Energia do período} / \text{tamanho do período} = 8 / 4 = 2$. (Potência finita!)

6.4. Simulação

O código Python abaixo implementa as equações numericamente para validar nossos cálculos analíticos. Para o limite tendendo ao infinito da potência, usaremos uma janela grande ($N = 1000$).

```
import numpy as np
import matplotlib.pyplot as plt
```

```

# -----
# 1. Análise do Sinal de Energia (Pulso Retangular)
# -----
n_energia = np.arange(-10, 15)
# Sinal x1[n] = 2 para 0 <= n <= 4, e 0 para os demais
x1 = np.where((n_energia >= 0) & (n_energia <= 4), 2.0, 0.0)

# Cálculo numérico da Energia (Soma dos quadrados)
E_num = np.sum(np.abs(x1)**2)

# Cálculo numérico da Potência no intervalo total analisado
# O len(n_energia) simula o 2N+1 do intervalo limitado
P_x1_estimada = E_num / len(n_energia)

# -----
# 2. Análise do Sinal de Potência (Senoide)
# -----
N_pot = 1000 # Intervalo de simulação -N até N
n_potencia = np.arange(-N_pot, N_pot + 1)
# Sinal x2[n] = 2 * sen(pi * n / 2)
x2 = 2.0 * np.sin(np.pi * n_potencia / 2.0)

# Cálculo da energia ao longo dessa janela gigantesca (E tenderá a
infinito)
E_x2_infinita = np.sum(np.abs(x2)**2)

# Cálculo numérico da Potência (Média da energia no intervalo 2N+1)
P_num = E_x2_infinita / (2 * N_pot + 1)

# -----
# Apresentação dos Resultados
# -----
plt.figure(figsize=(12, 6))

# Gráfico Sinal de Energia
plt.subplot(1, 2, 1)
plt.stem(n_energia, x1, linefmt='b-', markerfmt='bo', basefmt='k-')
plt.title('Sinal de Energia: Pulso Retangular')
plt.xlabel('n')
plt.ylabel('x1[n]')
plt.grid(True)

```

```

# Gráfico Sinal de Potência (Mostrando apenas um pequeno trecho para
visualização)
plt.subplot(1, 2, 2)
trecho = (n_potencia >= -5) & (n_potencia <= 5)
plt.stem(n_potencia[trecho], x2[trecho], linefmt='r-', markerfmt='ro',
basefmt='k-')
plt.title('Sinal de Potência: Senoide (Trecho visível)')
plt.xlabel('n')
plt.ylabel('x2[n]')
plt.grid(True)

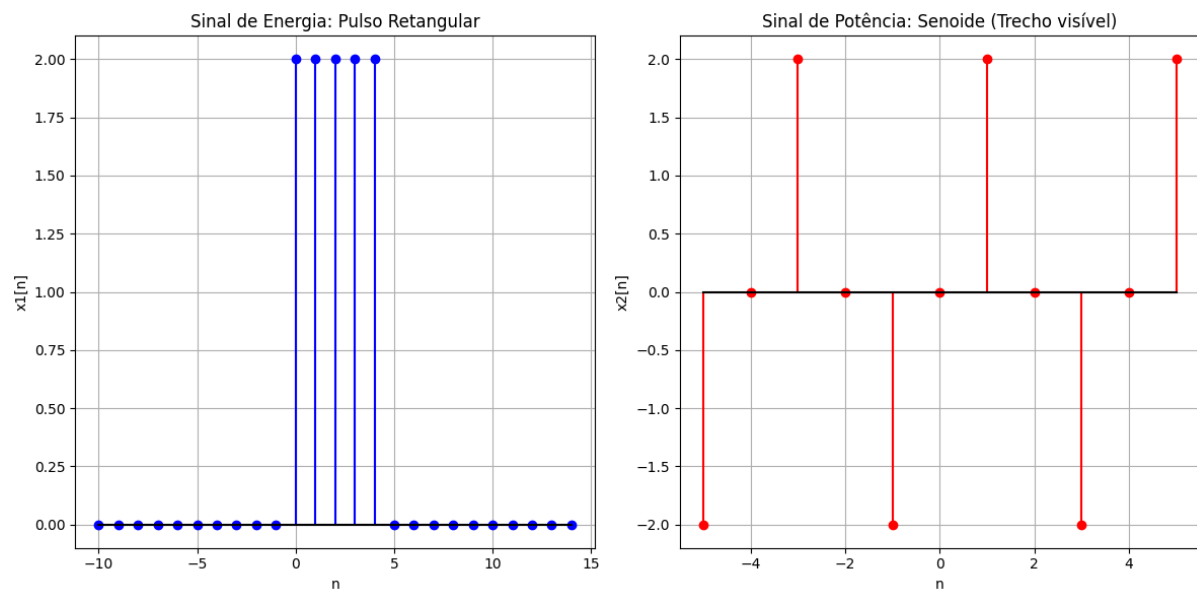
plt.tight_layout()
plt.show()

# Imprimindo verificações no terminal
print("--- Validação Numérica ---")
print(f"Sinal x1 (Energia) -> Energia Total = {E_num:.2f} | Potência
Estimada = {P_x1_estimada:.4f} (Tende a 0 se intervalo crescer)")
print(f"Sinal x2 (Potência) -> Energia na janela analisada =
{E_x2_infinita:.2f} (Explodindo para infinito) | Potência Média =
{P_num:.4f}")

```

6.5. Resultados

A saída no terminal do código Python será:



Validação Numérica:

Sinal x1 (Energia) -> Energia Total = 20.00 | Potência Estimada = 0.8000 (Tende a 0 se intervalo crescer)

Sinal $\times 2$ (Potência) $\rightarrow$ Energia na janela analisada = 4000.00 (Explodindo para infinito) | Potência Média = 1.9990

Os gráficos gerados mostram:

1. À esquerda: Um pulso contendo apenas 5 amostras não nulas de amplitude 2, restritas ao intervalo de $n=0$ a $n=4$, confirmando seu caráter transitório (não periódico).
2. À direita: Um trecho de uma onda senoidal que oscila continuamente entre os valores de -2 e $+2$, evidenciando seu padrão repetitivo e infinito.

6.6. Discussão

Os resultados computacionais batem perfeitamente com a fundamentação matemática.

No caso do pulso retangular (x_1), a energia se consolidou em exatamente 20, pois só existem 5 amostras ativas. Quando tentamos calcular a potência estimando com as amostras ao redor, o valor (0.80) já indica um número pequeno. Se aumentássemos a simulação de um intervalo de 25 posições para 1 milhão de posições (aumentando N), a energia continuaria sendo 20, mas dividiríamos por 1 milhão, o que zeraria a potência. Portanto, é classicamente um sinal de energia.

No caso da senoide (x_2), a energia acumulada na nossa janela de -1000 a 1000 (2001 amostras) bateu incríveis 4000. Se aumentarmos N para 1 milhão, a energia pulará para 2 milhões, explodindo ao infinito. Contudo, a relação entre essa energia gigantesca e o tamanho colossal da janela resulta em um valor perfeitamente constante: $P = 1.9990$ (matematicamente igual a 2.0 se pegarmos o limite real). Isso confirma que sinais contínuos e periódicos não podem ser medidos por sua "energia total" (pois ela é ilimitada), mas sim pela sua potência média, classificando-os de forma indiscutível como sinais de potência.

7.1. Sistema BIBO

Um sistema discreto é uma transformação matemática ou algoritmo (representado pelo operador T) que mapeia um sinal de entrada (excitação) em um sinal de saída (resposta). Para que possamos prever e controlar o comportamento desses sistemas na engenharia, nós os classificamos com base em cinco propriedades fundamentais:

- **Memória:** Um sistema é "sem memória" (ou estático) se a saída no instante atual depende única e exclusivamente do sinal de entrada no exato instante atual. Se o sistema precisar consultar informações do passado ou do futuro do sinal de entrada, dizemos que ele possui memória (é um sistema dinâmico).
- **Causalidade:** Um sistema é causal se ele não for capaz de prever o futuro. A saída só pode depender de instantes atuais ou passados da entrada. Em processamento em tempo real, todo sistema físico deve ser obrigatoriamente causal.

- Linearidade: Um sistema é linear se ele obedece ao princípio da superposição. Isso significa que, se combinarmos e escalonarmos vários sinais na entrada, a saída será exatamente a mesma combinação escalonada das saídas individuais. A linearidade simplifica enormemente a matemática, pois permite decompor sinais complexos.
-
- Invariância no Tempo: Um sistema é invariante no tempo se o seu comportamento intrínseco não muda com o passar do relógio. Se você atrasar a entrada em 5 segundos, a saída será exatamente a mesma de antes, apenas atrasada em 5 segundos.
- Estabilidade BIBO (Bounded-Input, Bounded-Output): Um sistema é estável se, ao receber um sinal de entrada limitado (que não explode para o infinito), ele garantir que o sinal de saída também será limitado. É um critério crucial para garantir que filtros e controladores não entrem em colapso.

7.2. Formulação matemática

A relação entrada-saída genérica é dada por:

$$y[n] = T\{x[n]\}$$

- Linearidade (Superposição):
 - Se $T\{x_1[n]\} = y_1[n]$ e $T\{x_2[n]\} = y_2[n]$, a linearidade exige que:
 - $T\{a * x_1[n] + b * x_2[n]\} = a * y_1[n] + b * y_2[n]$
- Invariância no Tempo:
 - Se a entrada $x[n]$ produz a saída $y[n]$, então a entrada deslocada $x[n - n_0]$ deve produzir a mesma saída deslocada:
 - $T\{x[n - n_0]\} = y[n - n_0]$
- Estabilidade BIBO:
 - Se existir um número finito B_x tal que $|x[n]| \leq B_x$ para todo n , então deve existir um número finito B_y tal que $|y[n]| \leq B_y$ para todo n .

7.3. Exemplo (Investigação Analítica)

Vamos classificar matematicamente os cinco sistemas propostos:

1. $y[n] = 2 * x[n]$

- Memória: Sem memória (depende apenas do instante n atual).
- Causalidade: Causal (não depende do futuro).
- Linearidade: Linear. $T\{a * x_1 + b * x_2\} = 2 * (a * x_1 + b * x_2) = a * (2 * x_1) + b * (2 * x_2)$.
- Invariância: Invariante. $T\{x[n - n_0]\} = 2 * x[n - n_0] = y[n - n_0]$.

- Estabilidade: Estável. Se $|x[n]| < B_x$, então $|y[n]| = 2|x[n]| < 2B_x$ (Finito).

2. $y[n] = x[n] + x[n - 1]$

- Memória: Com memória (depende da amostra anterior $n - 1$).
- Causalidade: Causal (depende do presente e passado).
- Linearidade: Linear. $(ax_1[n] + bx_2[n]) + (ax_1[n-1] + bx_2[n-1])$ agrupa em $ay_1[n] + by_2[n]$.
- Invariância: Invariante. $T\{x[n - n_0]\} = x[n - n_0] + x[(n - n_0) - 1] = y[n - n_0]$.
- Estabilidade: Estável. Se $|x|$ é limitado, a soma de dois valores limitados é limitada.

3. $y[n] = (x[n])^2$

- Memória: Sem memória (só depende de n).
- Causalidade: Causal.
- Linearidade: NÃO linear. $T\{a * x_1\} = (a * x_1)^2 = a^2 * x_1^2$. Diferente de $a * T\{x_1\}$ que seria $a * x_1^2$.
- Invariância: Invariante. $T\{x[n - n_0]\} = (x[n - n_0])^2 = y[n - n_0]$.
- Estabilidade: Estável. O quadrado de um número finito continua sendo finito.

4. $y[n] = x[n + 1]$

- Memória: Com memória (depende de um instante diferente do atual).
- Causalidade: NÃO causal (depende de $n + 1$, o futuro de n).
- Linearidade: Linear.
- Invariância: Invariante.
- Estabilidade: Estável.

5. $y[n] = n * x[n]$

- Memória: Sem memória (só requer o valor de x no próprio instante n).
- Causalidade: Causal.
- Linearidade: Linear. $n * (a * x_1 + b * x_2) = a(n * x_1) + b(n * x_2)$.

- Invariância: NÃO invariante. Um atraso no sinal de entrada gera $T\{x[n - n_0]\} = n * x[n - n_0]$. Porém, se atrasarmos a saída, teremos $y[n - n_0] = (n - n_0) * x[n - n_0]$. Como os dois resultados são diferentes, o sistema varia com o tempo.
- Estabilidade: NÃO estável. Mesmo que $x[n] = 1$ (sinal limitado em 1), a saída será $y[n] = n$. Quando o tempo n tender ao infinito, a saída tenderá ao infinito.

7.4. Simulação

O script Python a seguir injeta um sinal de entrada retangular básico em três desses sistemas para ilustrar visualmente as propriedades discutidas, com destaque para a falha de estabilidade do Sistema 5.

```
import numpy as np
import matplotlib.pyplot as plt

# Definindo o tempo n
n = np.arange(-5, 11)

# Sinal de entrada x[n]: Pulso de amplitude 1 entre n=0 e n=4
x = np.where((n >= 0) & (n <= 4), 1.0, 0.0)

# Simulação dos Sistemas
# Sistema 1: y[n] = 2x[n] (Ganho simples)
y1 = 2 * x

# Sistema 3: y[n] = x[n]^2 (Não Linear)
y3 = x**2

# Sistema 5: y[n] = n * x[n] (Variante no tempo e Não BIBO Estável)
y5 = n * x

# --- Plotagem dos Gráficos ---
plt.figure(figsize=(12, 10))

# 1. Sinal de Entrada
plt.subplot(4, 1, 1)
plt.stem(n, x, linefmt='k-', markerfmt='ko', basefmt='k-')
plt.title('Sinal de Entrada: x[n] (Limitado a amplitude 1)')
plt.ylabel('Amplitude')
plt.grid(True)

# 2. Sistema 1
plt.subplot(4, 1, 2)
```

```

plt.stem(n, y1, linefmt='b-', markerfmt='bo', basefmt='k-')
plt.title('Sistema 1 (Linear e Estável):  $y[n] = 2 * x[n]$ ')
plt.ylabel('Amplitude')
plt.grid(True)

# 3. Sistema 3
plt.subplot(4, 1, 3)
plt.stem(n, y3, linefmt='r-', markerfmt='ro', basefmt='k-')
plt.title('Sistema 3 (Não Linear):  $y[n] = (x[n])^2$ ')
plt.ylabel('Amplitude')
plt.grid(True)

# 4. Sistema 5
plt.subplot(4, 1, 4)
plt.stem(n, y5, linefmt='g-', markerfmt='go', basefmt='k-')
plt.title('Sistema 5 (Instável e Variante no Tempo):  $y[n] = n * x[n]$ ')
plt.xlabel('n')
plt.ylabel('Amplitude')
plt.grid(True)

plt.tight_layout()
plt.show()

# Demonstração numérica da Variância no tempo do Sistema 5
print("Prova de Variância no Tempo para  $y[n] = n * x[n]$  com  $n_0 = 2$ ")
print(" n | Entrada x[n] | Entrada Atrasada x[n-2] | T{x[n-2]} | Saida Atrasada y[n-2]")
print("-" * 80)
for i in range(5, 12): # Verificando n de 0 a 6
    # x[n-2] logicamente deslocado
    x_atrasado = 1 if 2 <= n[i] <= 6 else 0
    # APLICANDO o sistema na entrada atrasada:  $T\{x[n-2]\} = n * x[n-2]$ 
    T_x_atrasado = n[i] * x_atrasado
    # Atrasando a saida que já estava pronta:  $y[n-2] = (n-2) * x[n-2]$ 
    y_atrasado = (n[i] - 2) * x_atrasado
    print(f" {n[i]:1d} | {int(x[i])} | {T_x_atrasado} | {y_atrasado}")

```

7.5. Resultados

Os gráficos da simulação mostram o sinal de entrada retangular (altura 1, de $n=0$ a 4) sendo processado:

1. No Sistema 1, o pulso dobra de tamanho perfeitamente.
2. No Sistema 3, como o pulso tem altura 1, e $1^2 = 1$, o sinal de saída fica visualmente idêntico à entrada.
3. No Sistema 5, o pulso é transformado em uma rampa inclinada. Em $n=0$ ele vale 0, em $n=1$ vale 1, em $n=4$ vale 4.

A saída no terminal exibe a comparação numérica para o Sistema 5, mostrando que se aplicarmos o sistema em uma entrada que sofreu atraso de 2 amostras ($T\{x[n-2]\}$), os resultados divergem da saída pura deslocada ($y[n-2]$). Para $n=4$, o $T\{x[n-2]\}$ resulta em 4, enquanto o $y[n-2]$ resulta em 2.

7.6. Discussão

As simulações fundamentam as deduções matemáticas feitas anteriormente.

O comportamento do Sistema 5 ($y[n] = n * x[n]$) é o mais revelador. Primeiro, observamos graficamente a sua instabilidade BIBO: alimentamos o sistema com um sinal de entrada restrito e pacífico (cuja amplitude máxima é 1). No entanto, o sistema pega esse valor 1 e o multiplica pela variável do tempo infinito 'n'. Conforme 'n' cresce, o sinal de saída dispara, quebrando a condição de limite (Bounded-Output).

A análise no terminal também expôs fisicamente o que significa "invariância no tempo". Um sistema é invariante quando o próprio sistema não tem um "relógio interno" afetando o processo. No Sistema 5, o ganho de amplificação é o relógio atual (a variável n). Se o sinal chegar às 14h, ele sofre um ganho; se o mesmo sinal chegar atrasado às 16h, ele sofre outro ganho totalmente diferente. Por isso, a operação matemática $T\{x[n-2]\}$ produziu respostas diferentes de $y[n-2]$, classificando-o como um sistema variante no tempo. Sistemas lineares e invariantes no tempo (LTI - Linear Time-Invariant) são o foco dourado da engenharia, pois são previsíveis, diferentemente deste último.

8.1. O significado de linearidade

A linearidade em um sistema indica que ele obedece ao princípio da superposição, que se divide em duas propriedades fundamentais: aditividade e homogeneidade (escalonamento). Se um sistema recebe uma combinação linear de vários sinais de entrada, a sua saída será exatamente a mesma combinação linear das respostas individuais a cada um desses sinais. Matematicamente, se as entradas $x_1[n]$ e $x_2[n]$ produzem as saídas $y_1[n]$ e $y_2[n]$, o sistema é linear se, e somente se, a entrada $ax_1[n] + bx_2[n]$ produzir a saída $ay_1[n] + by_2[n]$, para quaisquer constantes a e b .

8.2. O significado de invariância no tempo

Um sistema é invariante no tempo quando suas características e seu comportamento interno não mudam com a passagem do tempo. Isso significa que um deslocamento temporal (atraso ou avanço) no sinal de entrada resultará em um deslocamento temporal idêntico no sinal de saída, sem alterar a forma ou a amplitude da resposta. Se a entrada $x[n]$ resulta na saída $y[n]$, então a entrada deslocada $x[n - n_0]$ resultará necessariamente na saída $y[n - n_0]$, para qualquer valor de n_0 .

8.3. O que é a resposta ao impulso A resposta ao impulso, denotada por $h[n]$, é o sinal de saída que um sistema produz quando o seu sinal de entrada é exatamente um impulso unitário (ou delta de Kronecker), representado por $\delta[n]$. O impulso unitário é um sinal que vale 1 no instante $n = 0$ e 0 em todos os outros instantes. Assim, a resposta ao impulso é definida pela relação: $h[n] = T\{\delta[n]\}$.

8.4. Por que a resposta ao impulso é importante Em sistemas que são simultaneamente Lineares e Invariantes no Tempo (LTI), a importância dessa classe está relacionada ao fato de que o sistema pode ser inteiramente caracterizado por sua resposta ao impulso. Ela funciona como o "DNA" do sistema. Conhecendo apenas o vetor $h[n]$, é possível prever e calcular qual será a saída do sistema para qualquer outro sinal de entrada imaginável, sem precisar conhecer o circuito interno do sistema físico.

8.5. Como um sinal pode ser representado utilizando impulsos deslocados

Qualquer sinal discreto genérico $x[n]$ pode ser compreendido como uma sequência de amostras individuais. Como o impulso unitário $\delta[n]$ vale 1 em $n = 0$ e 0 no resto, um impulso deslocado $\delta[n - k]$ valerá 1 apenas no instante $n = k$. Multiplicando esse impulso pelo valor do sinal naquele exato instante ($x[k]$), isolamos aquela amostra específica. Portanto, o sinal inteiro pode ser reconstruído somando todos esses impulsos isolados e escalonados, de menos a mais infinito:

$$x[n] = \sum (\text{para } k \text{ de } -\infty \text{ a } +\infty) \text{ de } x[k] * \delta[n - k]$$

8.6. Por que essa propriedade permite caracterizar sistemas LTI

A capacidade de representar qualquer sinal como uma soma de impulsos, combinada com as propriedades LTI, leva à operação da soma de convolução.

Quando aplicamos o operador do sistema T sobre a representação do sinal em impulsos, temos:

$$y[n] = T\{ \sum (k \text{ de } -\infty \text{ a } +\infty) \text{ de } x[k] * \delta[n - k] \}$$

Pela propriedade da **linearidade**, podemos tirar a soma e a constante escalar $x[k]$ para fora do operador, aplicando a transformação apenas no sinal base:

$$y[n] = \sum (k \text{ de } -\infty \text{ a } +\infty) \text{ de } x[k] * T\{\delta[n - k]\}$$

Pela propriedade da **invariância no tempo**, sabemos que se a resposta a $\delta[n]$ é $h[n]$, então a resposta ao impulso deslocado $\delta[n - k]$ é simplesmente a resposta ao impulso deslocada $h[n - k]$. Substituindo isso na equação, chegamos à fórmula final:

$$y[n] = \sum (k \text{ de } -\infty \text{ a } +\infty) \text{ de } x[k] * h[n - k]$$

Isso prova matematicamente que a saída $y[n]$ de qualquer sistema LTI é apenas a convolução do sinal de entrada $x[n]$ com a resposta ao impulso $h[n]$ do sistema, justificando por que $h[n]$ contém tudo o que precisamos saber sobre a transformação.

9.1. Explicação matemática da convolução

A convolução discreta é a operação fundamental que descreve a relação entre a entrada, a resposta ao impulso e a saída de um sistema Linear e Invariante no Tempo (LTI). Matematicamente, a saída $y[n]$ é obtida pela fórmula:

$$y[n] = \sum (\text{para } k \text{ de } -\infty \text{ a } +\infty) \text{ de } x[k] * h[n - k]$$

Esta equação descreve o processo em quatro etapas sequenciais:

1. **Inversão (Rebatimento):** A resposta ao impulso $h[k]$ é invertida no eixo do tempo (k), passando a ser $h[-k]$.
2. **Deslocamento:** O sinal invertido é deslocado de " n " posições ao longo do eixo k , resultando em $h[n - k]$. O valor de " n " define o instante de tempo da saída que está a ser calculado.
3. **Multiplicação:** Os sinais $x[k]$ e $h[n - k]$ são sobrepostos e os seus valores correspondentes em cada índice k são multiplicados ponto a ponto.
4. **Soma:** Todos os produtos resultantes da etapa anterior são somados, produzindo um único valor escalar para a saída $y[n]$ no instante n . O processo repete-se para cada valor de n .

9.2. Cálculo manual (Exemplo orientador)

Considerando os sinais $x[n] = \{1, 2, 1\}$ e $h[n] = \{1, 1\}$, assumimos que o primeiro elemento de cada sequência ocorre no instante $n = 0$.

O comprimento da saída será $N_y = 3 + 2 - 1 = 4$. Portanto, a saída existirá para $n = 0, 1, 2$ e 3 .

Aplicando a soma de convolução apenas para os termos não nulos:

- **Para $n = 0$:**
 - $y[0] = x[0]*h[0]$
 - $y[0] = 1 * 1 = 1$
- **Para $n = 1$:**
 - $y[1] = x[0]*h[1] + x[1]*h[0]$
 - $y[1] = 1 * 1 + 2 * 1 = 1 + 2 = 3$
- **Para $n = 2$:**
 - $y[2] = x[1]*h[1] + x[2]*h[0]$

- $y[2] = 2 * 1 + 1 * 1 = 2 + 1 = 3$
- **Para $n = 3$:**
- $y[3] = x[2] * h[1]$
- $y[3] = 1 * 1 = 1$

A sequência resultante é: $y[n] = \{1, 3, 3, 1\}$.

9.3, 4 e 5. Implementação, Comparação e Representação (Python)

O código abaixo utiliza a biblioteca NumPy para realizar a convolução e Matplotlib para visualizar os três sinais, validando os cálculos manuais.

```
import numpy as np
import matplotlib.pyplot as plt

# Sinais do Experimento 1
x = np.array([1, 2, 1])
h = np.array([1, 1])

# 3. Cálculo computacional da convolução
y = np.convolve(x, h)

# 4. Comparação
print("Resultados do Experimento 1:")
print(f"Sinal x[n]: {x}")
print(f"Sinal h[n]: {h}")
print(f"Sinal y[n] (Computacional): {y}")
# O resultado impresso será [1 3 3 1], idêntico ao cálculo manual.

# 5. Representação Gráfica
nx = np.arange(len(x))
nh = np.arange(len(h))
ny = np.arange(len(y))

plt.figure(figsize=(10, 8))

plt.subplot(3, 1, 1)
plt.stem(nx, x, linefmt='b-', markerfmt='bo', basefmt='k-')
plt.title('Sinal de Entrada x[n]')
plt.grid(True)

plt.subplot(3, 1, 2)
plt.stem(nh, h, linefmt='r-', markerfmt='ro', basefmt='k-')
```

```
plt.title('Resposta ao Impulso h[n]')
plt.grid(True)

plt.subplot(3, 1, 3)
plt.stem(ny, y, linefmt='g-', markerfmt='go', basefmt='k-')
plt.title('Sinal de Saída y[n] = x[n] * h[n]')
plt.grid(True)

plt.tight_layout()
plt.show()
```

9.6. Modificação e repetição do experimento

Para observar o impacto de uma alteração no sistema, manteremos a entrada $x[n] = \{1, 2, 1\}$ e substituiremos a resposta ao impulso por $h_2[n] = \{1, -1\}$. O comprimento da saída continua a ser $N_y = 4$.

Cálculo manual para $h_2[n] = \{1, -1\}$:

- $y_2[0] = x[0] \cdot h_2[0] = 1 \cdot 1 = 1$
- $y_2[1] = x[0] \cdot h_2[1] + x[1] \cdot h_2[0] = 1 \cdot (-1) + 2 \cdot 1 = 1$
- $y_2[2] = x[1] \cdot h_2[1] + x[2] \cdot h_2[0] = 2 \cdot (-1) + 1 \cdot 1 = -1$
- $y_2[3] = x[2] \cdot h_2[1] = 1 \cdot (-1) = -1$

Nova sequência de saída: $y_2[n] = \{1, 1, -1, -1\}$.

9.7. Discussão sobre a alteração da resposta ao impulso

A alteração de $h[n]$ demonstrou que a resposta ao impulso define inteiramente o papel físico do sistema LTI sobre o sinal.

No primeiro caso ($h[n] = \{1, 1\}$), o sistema somou amostras consecutivas. Esta operação atua como um **filtro passa-baixo** (ou uma média móvel não normalizada), que tende a atenuar variações bruscas e produzir um sinal de saída mais suave e largo em duração.

No segundo caso ($h_2[n] = \{1, -1\}$), o sistema calculou a diferença matemática entre a amostra atual e a amostra anterior. Esta operação atua como um **filtro passa-alto** (ou detetor de bordas). O sistema bloqueou a componente média do sinal (o valor "estável") e realçou as transições, resultando numa saída que evidencia onde o sinal estava a subir (valores positivos 1, 1) e onde começou a descer (valores negativos -1, -1).

10.1. O Filtro de Média Móvel como Sistema LTI

O filtro de média móvel é um sistema Linear e Invariante no Tempo (LTI) amplamente utilizado para suavizar sinais e atenuar ruídos de alta frequência. A sua resposta ao impulso $h[n]$ é constituída por M coeficientes iguais, todos com o valor de $1/M$. A operação de

convolução $y[n] = x[n] * h[n]$ faz com que cada amostra da saída seja a média aritmética das M amostras mais recentes da entrada.

10.2. Implementação e Simulação (Python)

O código abaixo gera um sinal original em formato de pulso retangular, adiciona ruído aleatório (simulando interferência) e aplica o filtro de média móvel para os três casos solicitados ($M = 3$, $M = 5$ e $M = 10$).

```
import numpy as np
import matplotlib.pyplot as plt

# 1. Geração do Sinal Original e Contaminado
n = np.arange(0, 60)
# Sinal original: Pulso que vale 1 entre n=10 e n=40
x_original = np.where((n >= 10) & (n <= 40), 1.0, 0.0)

# Adicionando ruído gaussiano (média 0, desvio padrão 0.2)
np.random.seed(42) # Para reprodutibilidade
ruído = np.random.normal(0, 0.2, len(n))
x_contaminado = x_original + ruído

# 2. Definição das Respostas ao Impulso (h[n])
M_vals = [3, 5, 10]
h = {M: np.ones(M) / M for M in M_vals}

# 3. Filtragem por Convolução
y = {M: np.convolve(x_contaminado, h[M]) for M in M_vals}

# 4. Representação Gráfica
plt.figure(figsize=(12, 12))

# Plot do Sinal Original e Contaminado
plt.subplot(4, 1, 1)
plt.plot(n, x_original, 'k--', label='Original', linewidth=2)
plt.stem(n, x_contaminado, linefmt='r-', markerfmt='r.', basefmt='k-',
label='Contaminado')
plt.title('Sinal Original e Contaminado com Ruído')
plt.legend()
plt.grid(True)

# Plot dos Sinais Filtrados
cores = {3: 'b', 5: 'g', 10: 'm'}
for i, M in enumerate(M_vals, 2):
    plt.subplot(4, 1, i)
    plt.plot(n, y[M], color=cores[M], label=f'Filtrado (M={M})')
    plt.grid(True)
```

```

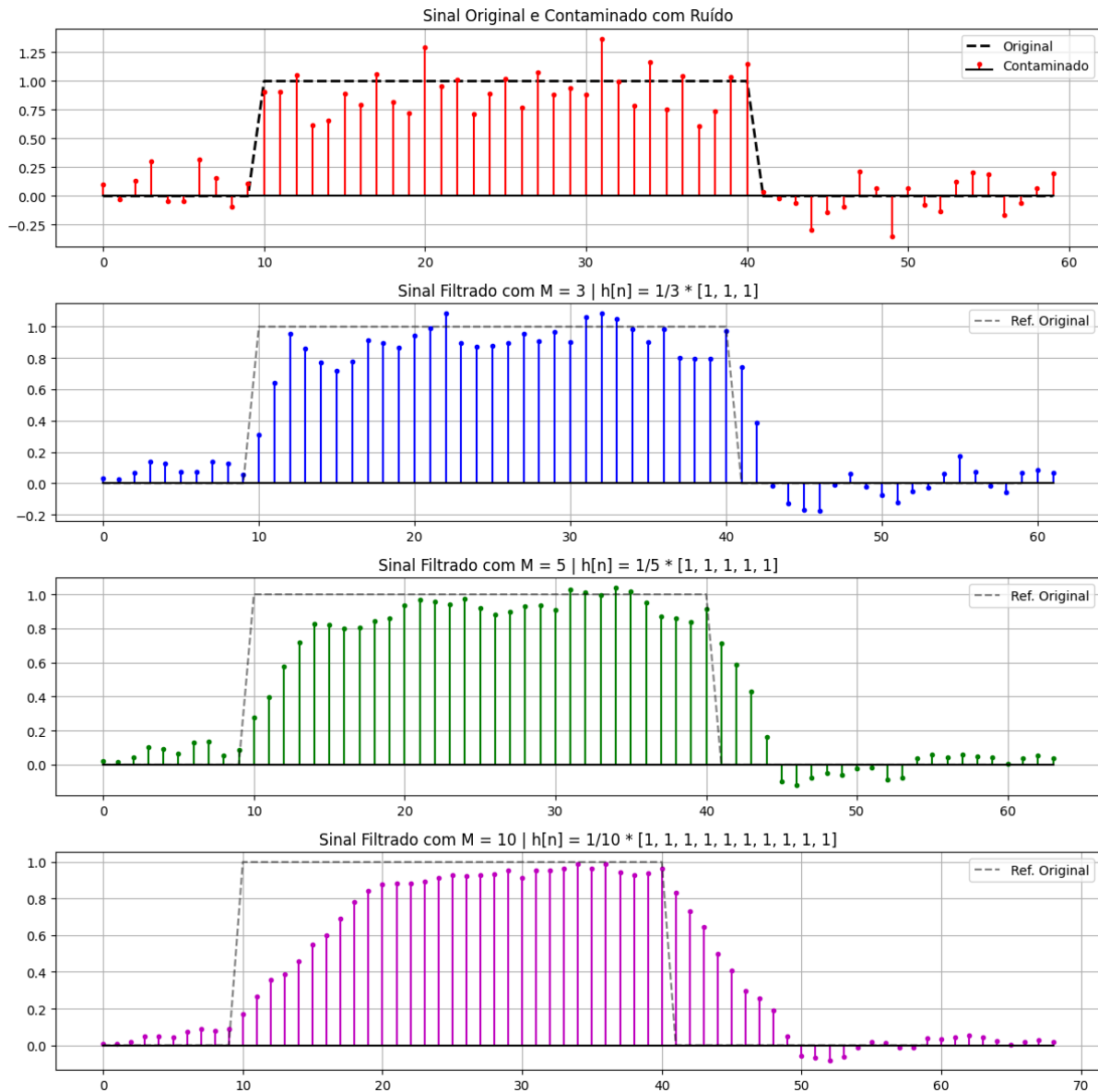
ny = np.arange(len(y[M]))

# Exibe a resposta ao impulso no canto do gráfico como texto
hn_str = f"h[n] = 1/{M} * [{', '}.join(['1']*M)]"

plt.stem(ny, y[M], linefmt=f'{cores[M]}-',
markerfmt=f'{cores[M]}.', basefmt='k-')
plt.plot(n, x_original, 'k--', alpha=0.5, label='Ref. Original') #
Referência
plt.title(f'Sinal Filtrado com M = {M} | {hn_str}')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()

```



10.3. Discussão dos Resultados

A análise dos gráficos gerados pela simulação permite observar o comportamento do sistema para cada valor de M :

- **Sinal Original e Contaminado:** O sinal original possui transições abruptas e exatas. A inserção do ruído provoca flutuações rápidas (altas frequências) em torno da amplitude verdadeira do sinal.
- **$M = 3$:** A resposta ao impulso possui apenas 3 coeficientes ($1/3$). O sinal filtrado apresenta uma ligeira redução na amplitude do ruído, mas a forma geral do pulso (incluindo as bordas de subida e descida) permanece bastante próxima à do sinal original. O atraso inserido é mínimo.

- **M = 5:** Com 5 coeficientes (1/5), a suavização do ruído torna-se mais evidente. As flutuações no patamar superior do pulso são menores. Contudo, as transições que antes eram verticais começam a ficar inclinadas (borradas), e o sinal sofre um atraso perceptível.
- **M = 10:** Com 10 coeficientes (1/10), o ruído é fortemente atenuado, deixando os patamares do sinal muito lisos. Em contrapartida, o pulso original é severamente distorcido no domínio do tempo. O início da subida é suave e só atinge a amplitude máxima muito depois do sinal original.

10.4. O Compromisso (Trade-off) entre Suavização e Alteração Temporal

A investigação evidencia um dos princípios mais fundamentais do processamento de sinais: existe um compromisso direto entre a capacidade de remover ruído e a preservação das características temporais do sinal.

Ao aumentar o número de coeficientes M , o filtro de média móvel incorpora mais amostras do passado para calcular o valor presente. Isso melhora a **suavização** (maior rejeição de altas frequências e eliminação de picos de ruído), pois o ruído aleatório tende a cancelar-se numa média longa.

No entanto, essa longa memória causa uma **alteração das características temporais**. Sinais com variações rápidas, como bordas e degraus, são borrados, transformando transições abruptas em rampas lentas. Além disso, o aumento de M introduz um "atraso de grupo" maior ao sistema. Matematicamente, a convolução introduz um espalhamento no tempo onde o comprimento do sinal de saída é $N_x + M - 1$. Quanto maior o M , mais tempo o sistema demora a "perceber" e a estabilizar após uma mudança na entrada, corrompendo a precisão temporal da informação.

Mini Projeto Integrador: Simulação de um Sistema de Aquisição e Processamento de Sinais

Fenômeno Escolhido: Batimento Cardíaco (Eletrocardiograma - ECG simplificado)

Etapa 1 – Modelagem do Sinal

Para representar o batimento cardíaco humano de forma matematicamente tratável usando as ferramentas vistas, utilizaremos a Série de Fourier truncada. Um batimento cardíaco é um sinal periódico. Assumindo uma frequência cardíaca de 72 batimentos por minuto (bpm), a frequência fundamental do fenômeno é $f_0 = 1,2$ Hz.

O modelo matemático para o sinal elétrico do coração $x(t)$, medido em milivolts (mV), será uma combinação da frequência fundamental com seus dois primeiros harmônicos para simular o formato de pico do complexo QRS:

$$x(t) = A_1 * \cos(2 * \pi * f_1 * t) - A_2 * \cos(2 * \pi * f_2 * t) + A_3 * \cos(2 * \pi * f_3 * t)$$

Significado físico dos parâmetros:

- **$f_1 = 1,2$ Hz:** Frequência fundamental (ritmo cardíaco básico de 72 bpm).

- **f2 = 2,4 Hz e f3 = 3,6 Hz:** Harmônicos que constroem a forma aguda do pulso elétrico ventricular.
- **A1 = 1.0 mV, A2 = 0.5 mV, A3 = 0.3 mV:** Amplitudes elétricas captadas pelos eletrodos, simulando a variação de potencial elétrico no músculo cardíaco.

Etapas 2 a 7 – Geração, Amostragem, Quantização, Ruído e Processamento

O script em Python a seguir contempla todas as etapas de simulação, desde a geração do sinal contínuo (em alta resolução temporal) até a filtragem por convolução.

```
import numpy as np
import matplotlib.pyplot as plt

# --- Etapa 2: Geração computacional (Sinal "Contínuo") ---
f0 = 1.2
t_cont = np.arange(0, 2.5, 0.001) # Alta resolução simulando tempo contínuo (2.5 segundos)
# Modelo: x(t) = 1.0*cos(2*pi*1.2*t) - 0.5*cos(2*pi*2.4*t) + 0.3*cos(2*pi*3.6*t)
x_cont = 1.0 * np.cos(2 * np.pi * f0 * t_cont) - 0.5 * np.cos(2 * np.pi * 2 * f0 * t_cont) + 0.3 * np.cos(2 * np.pi * 3 * f0 * t_cont)

# --- Etapa 3: Amostragem ---
fs = 100 # Frequência de amostragem de 100 Hz
Ts = 1 / fs
n = np.arange(0, 2.5, Ts) # Instantes discretos
x_n = 1.0 * np.cos(2 * np.pi * f0 * n) - 0.5 * np.cos(2 * np.pi * 2 * f0 * n) + 0.3 * np.cos(2 * np.pi * 3 * f0 * n)

# --- Etapa 4: Quantização ---
def quantizar(sinal, bits):
    n_niveis = 2**bits
    val_min, val_max = np.min(sinal), np.max(sinal)
    passo = (val_max - val_min) / (n_niveis - 1)
    return np.round((sinal - val_min) / passo) * passo + val_min

x_4bits = quantizar(x_n, 4) # 16 níveis
x_8bits = quantizar(x_n, 8) # 256 níveis

# --- Etapa 5: Inclusão de ruído ---
# Ruído gaussiano (simulando artefatos musculares de alta frequência e contato do eletrodo)
np.random.seed(42)
```

```

ruído = np.random.normal(0, 0.2, len(x_n))
x_ruído = x_8bits + ruído # Aplicamos o ruído sobre o sinal
digitalizado (8 bits)

# --- Etapa 6: Processamento por Sistema LTI (Filtro FIR de Média
Móvel) ---
M = 5 # Filtro de média móvel com 5 amostras (suavização de 50 ms)
h = np.ones(M) / M
y = np.convolve(x_ruído, h, mode='same') # Convolução discreta
(mode='same' mantém o eixo de tempo alinhado)

# --- Etapa 7: Comparação dos Resultados (Plotagem) ---
plt.figure(figsize=(14, 16))

# 1. Sinal Contínuo vs Amostrado
plt.subplot(5, 1, 1)
plt.plot(t_cont, x_cont, 'b-', alpha=0.5, label='Contínuo x(t)')
plt.stem(n, x_n, linefmt='r-', markerfmt='r.', basefmt='k-',
label='Amostrado x[n]')
plt.title('Etapa 3: Sinal Contínuo e Amostragem (fs = 100 Hz)')
plt.legend()
plt.grid(True)

# 2. Efeito da Quantização
plt.subplot(5, 1, 2)
plt.step(n, x_4bits, 'g-', label='Quantizado (4 bits - 16 níveis)',
where='mid')
plt.plot(n, x_8bits, 'r--', label='Quantizado (8 bits - 256 níveis)',
alpha=0.8)
plt.title('Etapa 4: Quantização (4 bits vs 8 bits)')
plt.legend()
plt.grid(True)

# 3. Sinal Original vs Ruído
plt.subplot(5, 1, 3)
plt.plot(n, x_8bits, 'k-', linewidth=2, label='Sinal de Interesse (8
bits)')
plt.plot(n, ruído, 'm-', alpha=0.5, label='Ruído Aleatório')
plt.title('Etapa 5: Sinal Original e Ruído Adicionado')
plt.legend()
plt.grid(True)

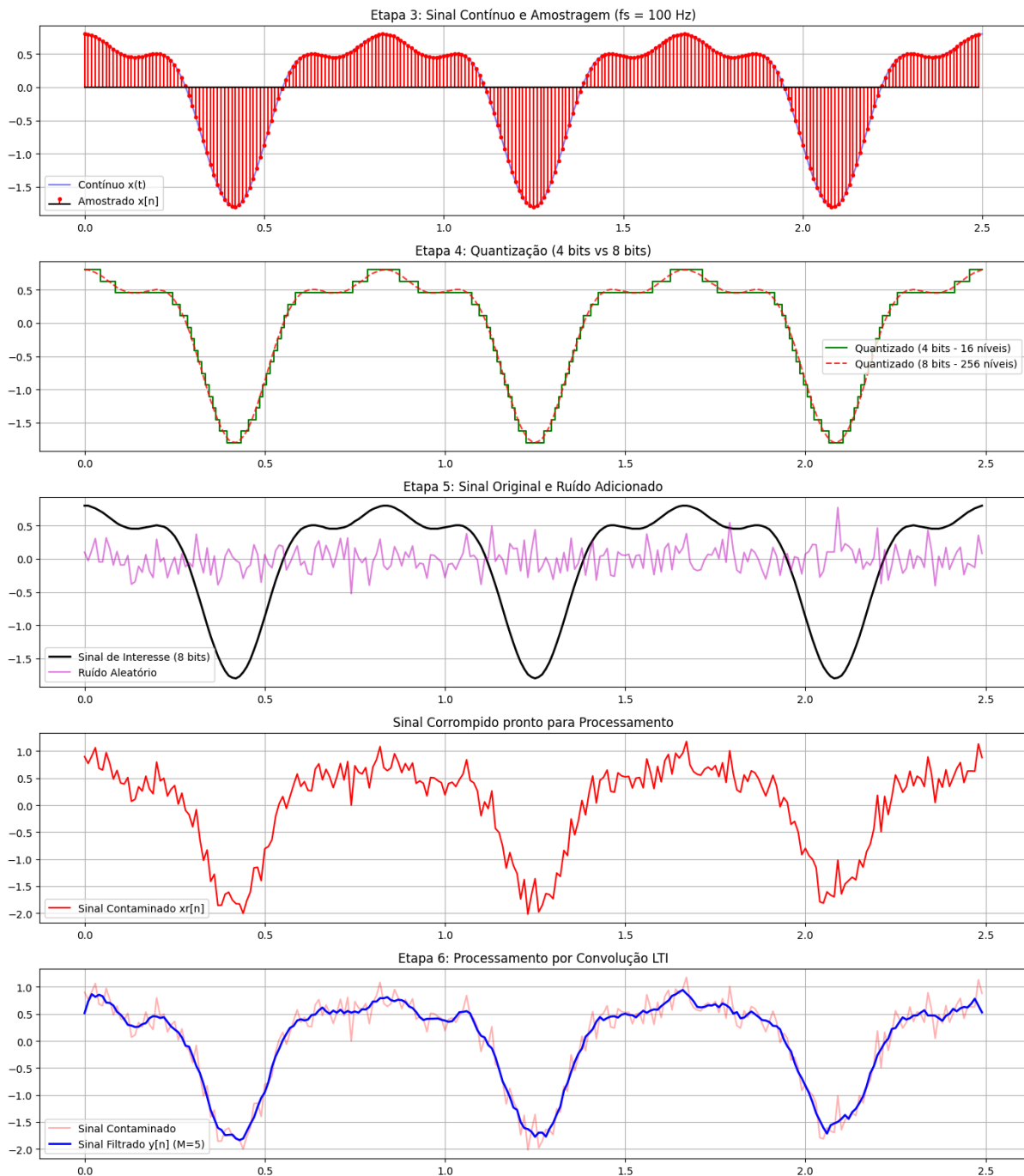
# 4. Sinal Contaminado

```

```
plt.subplot(5, 1, 4)
plt.plot(n, x_ruído, 'r-', label='Sinal Contaminado xr[n]')
plt.title('Sinal Corrompido pronto para Processamento')
plt.legend()
plt.grid(True)

# 5. Filtragem por Convolução
plt.subplot(5, 1, 5)
plt.plot(n, x_ruído, 'r-', alpha=0.3, label='Sinal Contaminado')
plt.plot(n, y, 'b-', linewidth=2, label=f'Sinal Filtrado y[n] (M={M})')
plt.title('Etapa 6: Processamento por Convolução LTI')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()
```



Etapa 8 – Discussão

1. Como o fenômeno foi representado matematicamente?

Foi representado pela soma de três ondas cosseno (frequência fundamental de 1,2 Hz e dois harmônicos). Esta superposição cria picos periódicos que emulam de forma simplificada a característica oscilatória e aguda do complexo QRS de um eletrocardiograma.

2. Como a escolha de f_s modificou a representação do sinal?

A frequência de amostragem ($f_s = 100$ Hz) resultou em um intervalo $T_s = 0,01$ s. Como a maior frequência do sinal modelo é 3,6 Hz, $f_s = 100$ Hz respeita folgadoamente o Teorema de

Nyquist ($f_s > 2 * f_{\text{max}}$), permitindo que o sinal discreto capte perfeitamente o formato do "pico" do coração sem serrilhamentos excessivos (aliasing).

3. Qual foi o efeito da quantização?

A quantização arredondou as amplitudes exatas do sinal amostrado para um conjunto discreto de degraus de tensão, introduzindo um erro conhecido como ruído de quantização (truncamento).

4. Como o número de bits afetou o resultado?

Com 4 bits, havia apenas 16 níveis de tensão disponíveis; o sinal resultante formou "degraus" largos e visíveis, distorcendo severamente a morfologia do sinal cardíaco. Com 8 bits (256 níveis), o tamanho do degrau tornou-se tão diminuto que o sinal digitalizado ficou visualmente idêntico ao sinal original, provando que maior resolução minimiza o erro de quantização.

5. Qual foi o efeito do ruído?

A adição de um ruído gaussiano (média 0, desvio padrão 0.2 mV) emulou a interferência elétrica e o tremor muscular (artefatos miográficos) comuns em sensores reais. Ele transformou a linha de base lisa do ECG em um traçado espesso e instável.

6. Qual a resposta ao impulso do sistema utilizado?

A resposta ao impulso foi um vetor $h[n]$ contendo $M=5$ coeficientes com valor $1/5$, ou seja, $h[n] = \{0.2, 0.2, 0.2, 0.2, 0.2\}$.

7. Por que o sistema utilizado pode ser considerado LTI?

O filtro de média móvel é Linear pois obedece ao princípio da superposição (a média da soma de dois sinais é igual à soma de suas médias) e é Invariante no Tempo pois os coeficientes do filtro não mudam dependendo de quando o batimento cardíaco ocorre.

8. Qual foi o efeito da convolução?

A convolução deslizou o vetor $h[n]$ sobre o sinal contaminado $xr[n]$, calculando a média aritmética de janelas de 5 amostras móveis (equivalente a uma janela de 50 milissegundos).

9. O sistema reduziu as variações introduzidas pelo ruído?

Sim. Como o ruído gaussiano é aleatório e de alta frequência espacial, ele oscila rapidamente entre valores positivos e negativos. A média móvel forçou o cancelamento dessas variações espúrias, limpando a linha de base do ECG.

10. Houve alteração significativa do sinal de interesse?

Sim, ocorreu um trade-off previsível (compromisso). Embora a base tenha ficado mais limpa, os picos agudos do "batimento cardíaco" (simulando a onda R) sofreram uma ligeira atenuação (ficaram mais baixos e um pouco mais largos).

11. Quais limitações foram observadas?

A média móvel trata todas as amostras com o mesmo peso e causa atraso temporal de grupo. O modelo matemático foi bastante simples (série de Fourier rudimentar), não captando perfeitamente as complexidades das ondas P e T de um coração humano real.

12. Como o modelo poderia ser melhorado?

Para a geração, poder-se-ia modelar o sinal de ECG através de equações paramétricas dinâmicas que sintetizam exatamente os tempos e amplitudes das ondas P, Q, R, S, T. Para o processamento, o filtro de média móvel poderia ser substituído por um Filtro FIR passa-baixa projetado com janela de Hamming, que atenua o ruído de alta frequência sem destruir e achatar tão agressivamente o pico cardíaco.

Conclusão

Este relatório consolidou os fundamentos teóricos e práticos do Processamento Digital de Sinais (PDS) por meio de uma abordagem integrada entre modelagem matemática e simulação computacional. A exploração das operações básicas sobre sequências discretas e a classificação de sinais baseada em energia e potência estabeleceram o alicerce para a compreensão profunda dos sistemas discretos.

A análise das propriedades dos sistemas, com ênfase na linearidade e na invariância no tempo (LTI), demonstrou como a resposta ao impulso e a convolução são ferramentas indispensáveis para prever o comportamento de filtros e transformar informações. Toda essa base teórica foi materializada no mini projeto integrador, que simulou a cadeia completa de aquisição de um sinal eletrocardiográfico (ECG). As etapas sequenciais de amostragem, quantização, adição de ruído e filtragem por média móvel revelaram, na prática, os impactos da resolução digital e o clássico compromisso da engenharia entre a atenuação de interferências e a preservação das características temporais essenciais do sinal de interesse.

Referência Bibliográfica

LATHI, B. P. Sinais e Sistemas Lineares. Porto Alegre: Bookman.